

2^η Εργασία

Αναγνώρισης Προτύπων Στατιστική Μάθηση

Πριμεράκης Αχιλλεύς

Κώδικας:

Για την υλοποίηση αυτής της εργασίας ακολούθησα τα ίδια αρχικά βήματα με την προηγούμενη, δηλαδή συμπεριέλαβα τις απαραίτητες βιβλιοθήκες και έπειτα διάβασα τα αρχεία του dataset MNIST. Στις παρακάτω σελίδες παραθέτω στιγμιότυπα από τον κώδικα, τα αποτελέσματα για διαφορετικές υπερπαραμέτρους, καμπύλες εκπαίδευσης, χαρακτηριστικά παραδείγματα ορθής και εσφαλμένης ταξινόμησης και πίνακες σύγχυσης. Για κάθε έναν από τους τρεις γραμμικούς ταξινομητές υπάρχει η Α φάση όπου λαμβάνω 1000 δείγματα εκπαίδευσης ανά κλάση και η Β φάση όπου λαμβάνω 1000 δείγματα εκπαίδευσης για μία κλάση και 50 δείγματα εκπαίδευσης για κάθε μία από τις υπόλοιπες κλάσεις.

Βιβλιοθήκες

```
import pandas as pd
import numpy as np
from PIL import Image
from io import BytesIO
import statistics
import time
import matplotlib.pyplot as plt
```

✓ 1.6s

Διάβασμα αρχείων

```
parquet_file = r'C:\Users\user\Desktop\pythonikos\train-00000-of-00001.parquet'
df = pd.read_parquet(parquet_file, engine='auto')

parquet_file = r'C:\Users\user\Desktop\pythonikos\test-00000-of-00001.parquet'
df_test = pd.read_parquet(parquet_file, engine='auto')

df.head()
```

✓ 0.3s

Μετατροπή εικόνων σε vectors

```
train_set_synolo = 60000
test_set_synolo = 10000

topiki_boithitiki_lista_train = []
labels_lista_train = []

for i in range (train_set_synolo):
    raw_bytes = df.iloc[i]['image']['bytes']

    img = Image.open(BytesIO(raw_bytes))
    img = img.convert('L')

    arr = np.array(img)
    vector = arr.reshape(-1)
    topiki_boithitiki_lista_train.append(vector)

    labels_lista_train.append(df.iloc[i]['label'])

X_train = np.array(topiki_boithitiki_lista_train)
X_train = X_train.astype('float32') / 255.0

topiki_boithitiki_lista_test = []
labels_lista_test = []

for i in range (test_set_synolo):
    raw_bytes = dftest.iloc[i]['image']['bytes']

    img = Image.open(BytesIO(raw_bytes))
    img = img.convert('L')

    arr = np.array(img)
    vector = arr.reshape(-1)
    topiki_boithitiki_lista_test.append(vector)

    labels_lista_test.append(dftest.iloc[i]['label'])

X_test = np.array(topiki_boithitiki_lista_train)
X_test = X_train.astype('float32') / 255.0
```

✓ 9.1s

Δημιουργία των 45 ζευγαριών σύγκρισης δηλαδή 0 vs 1, 0vs2, . . . , 8 vs 9.

```

comparison = []

for i in range(10):
    for j in range(i,10):
        if i != j:
            comparison.append([i, j])

# print(comparison)
# print('megethos',len(comparison))

```

✓ 0.0s

Απόφασης

```

def apofasi(a):
    if a <= 0:
        return -1
    return 1

```

✓ 0.0s

Συλλογή 1000 δειγμάτων ανά κατηγορία 0...9.

```

lista_me_10_trains = []

for i in range(10):
    topiki_boithitiki_lista_kategorias = []

    j = 0

    while len(topiki_boithitiki_lista_kategorias) < 1000:
        if i == labels_lista_train[j]:
            topiki_boithitiki_lista_kategorias.append(X_train[j])
            j+=1
    lista_me_10_trains.append(topiki_boithitiki_lista_kategorias)

```

✓ 0.0s

Πρώτη φάση 1 V 1

Παρακάτω βρίσκω τα weights των 45 ζευγαριών που συγκρίνουμε. Αρχικά παίρνω τους 2 χαρακτήρες και τοποθετώ σαν labels τους το 1 με αναφορά στο πρώτο και το -1. Για την αποφυγή του overfitting αποφάσισα πως σε κάθε γύρω εκπαίδευσης θα κάνω shuffling, τυχαίο ανακάτεμα. Έπειτα βάση του ποσίου του εσωτερικού γινομένου του δείγματος με το βάρος καταλαβαίνω αν η πρόβλεψη δεν ήταν ακριβής. Σε περίπτωση που έχουμε False Negative δηλαδή το δείγμα ήταν αυτό που ψάχναμε και δεν το βρήκαμε τότε προσθέτουμε τα pixels του στο βάρος, αν είναι False Positive τότε αφαιρούμε τα pixels.

```
weights = []

for i in range(45):
    topiki_boithitiki_barwn = np.zeros(784)

    a = comparison[i][0]
    b = comparison[i][1]

    eikones_a = lista_me_10_trains[a]
    eikones_b = lista_me_10_trains[b]

    combined_a_and_b = np.array(eikones_a + eikones_b)

    combined_a_and_b_labels = np.array([1] * 1000 + [-1] * 1000)

    anakatema = np.arange(2000)

    np.random.shuffle(anakatema)

    combined_a_and_b_anakatemena = combined_a_and_b[anakatema]

    combined_a_and_b_labels_anakatemena = combined_a_and_b_labels[anakatema]

    for j in range(2000):
        sum = 0

        for z in range(784):
            sum += combined_a_and_b_anakatemena[j][z] * topiki_boithitiki_barwn[z]

        if apofasi(sum) == -1 and combined_a_and_b_labels_anakatemena[j] == 1:
            for z in range(784):
                topiki_boithitiki_barwn[z] += combined_a_and_b_anakatemena[j][z]
        elif apofasi(sum) == 1 and combined_a_and_b_labels_anakatemena[j] == -1:
            for z in range(784):
                topiki_boithitiki_barwn[z] -= combined_a_and_b_anakatemena[j][z]
```

✓ 29.8s

Τέλος βγαίνει η απόφαση σε ποια κατηγορία ανήκει το νούμερο βάση μιας ψηφοφορίας των 45 ταξινομητών.

```

lista_gia_test = X_test[0:2000]

lista_gia_test_labels = labels_lista_test[0:2000]
#print(len(lista_gia_test))

sum_epityxiwn = 0

for deigma_test_idx in range(2000):
    topiki_boithitiki_psifwn = np.zeros(10)

    for weight_table_idx in range(45):
        sum = 0

        sum = np.dot(weights[weight_table_idx], lista_gia_test[deigma_test_idx])

        if apofasi(sum) == 1:
            topiki_boithitiki_psifwn[comparison[weight_table_idx][0]] +=1
        else:
            topiki_boithitiki_psifwn[comparison[weight_table_idx][1]] +=1

    if lista_gia_test_labels[deigma_test_idx] == np.argmax(topiki_boithitiki_psifwn):
        sum_epityxiwn +=1

print(sum_epityxiwn)

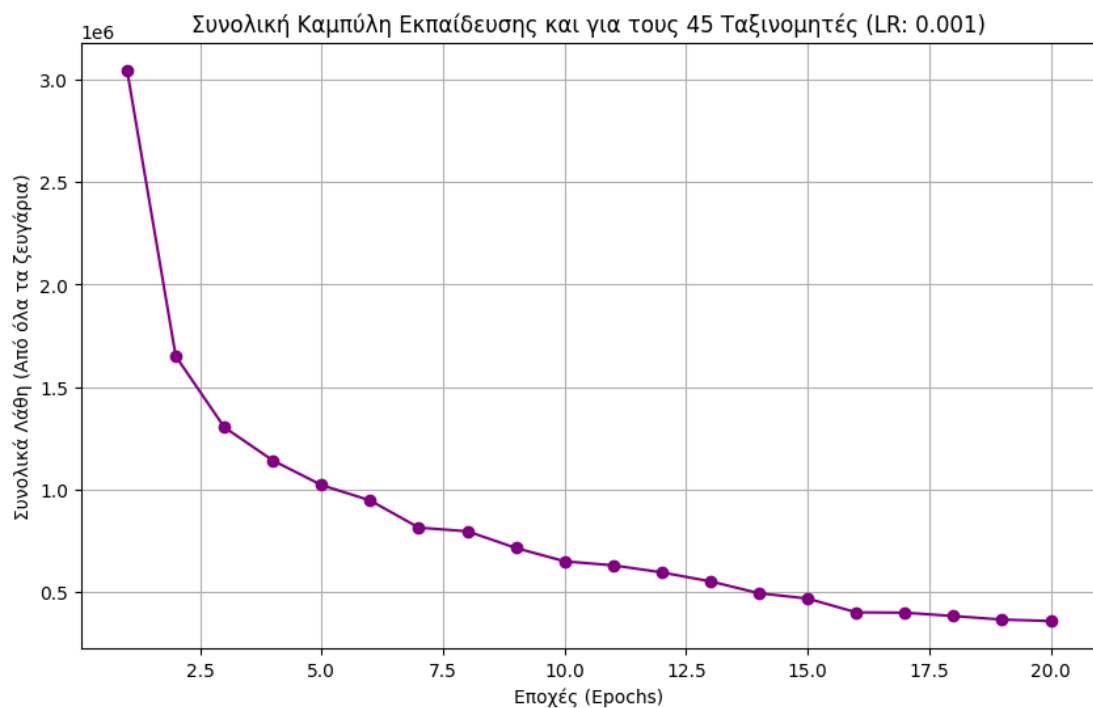
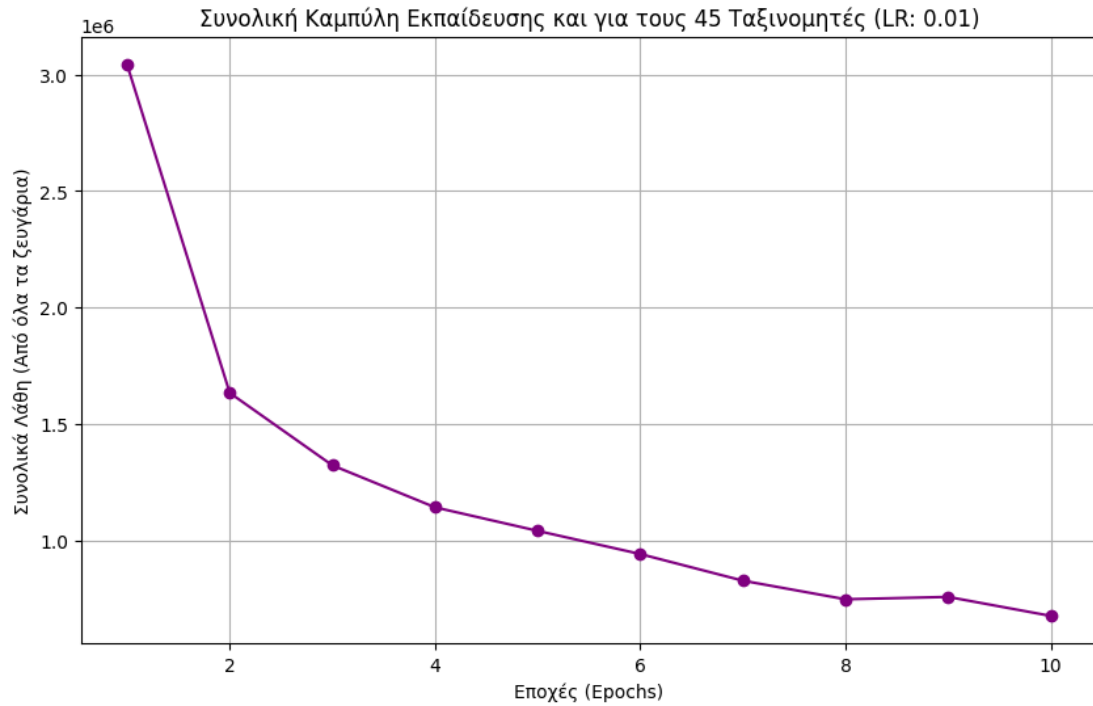
```

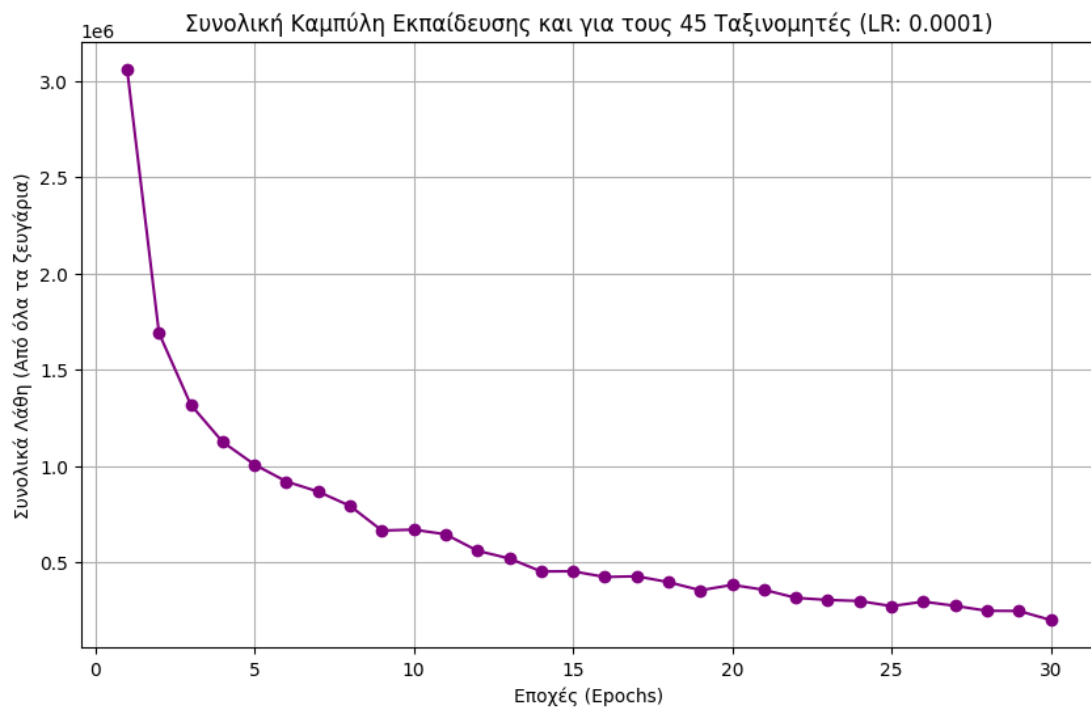
Learning rate	Epochs	Επιτυχίες στα 2000	Χρόνος
1	1	1798	2.4 sec
0.1	1	1782	1.8 sec
0.01	1	1798	1.9 sec
0.001	1	1779	1.8 sec
0.0001	1	1765	2 sec
1	10	1905	9.5 sec
1	20	1933	14.9 sec
1	30	1966	20.5 sec
0.1	10	1917	0.6 sec
0.1	20	1937	14.8 sec
0.1	30	1927	15.2 sec
0.01	10	1903	9.4 sec
0.01	20	1958	15 sec
0.01	30	1971	19.3 sec
0.0001	10	1906	9.2 sec
0.0001	20	1932	15 sec
0,0001	30	1978	21.3 sec

Όπως καταλαβαίνουμε από τα παραπάνω αποτελέσματα η αύξηση των εποχών εκπαίδευσης έχει πολύ σημαντικό ρόλο στην αύξηση της ακρίβειας. Επιπλέον εάν δεν χρησιμοποιήσουμε epochs υπάρχει μεγάλη απόκλιση σε διαφορετικές εκτελέσεις ίδιου κώδικα λόγω του shuffling. Παρόλα αυτά το learning

rate δεν έχει τόσο μεγάλη επιρροή όσο και τα epochs παρόλο που και αυτό δίνει μια σταθερότητα.

Οι καμπύλες έχουν παρόμοια πτώση και οι τρείς με μεγάλη πτώση και σταθεροποίηση μετά τις 7 εποχές,





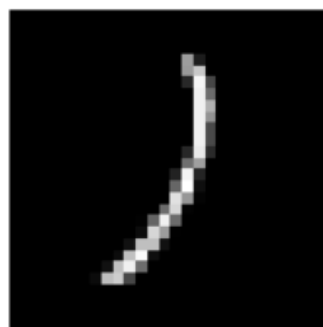
Παραδείγματα:

Στα λάθη γραφικός ο χαρακτήρας είναι πολύ ανακριβείς ακόμα και για το ανθρώπινο μάτι. Στα σωστά το μοντέλο κατάφερε να αναγνωρίσει και δύσκολες περιπτώσεις.

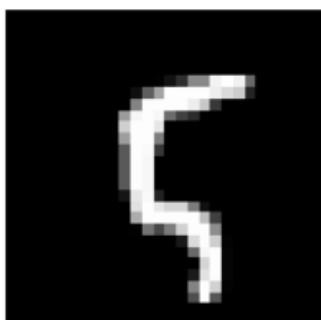
Real: 2, Pred: 2



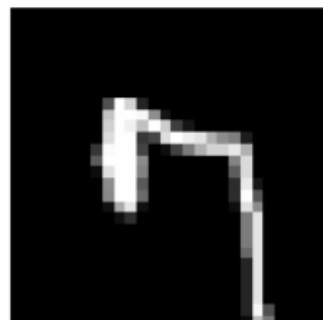
Real: 1, Pred: 2



Real: 5, Pred: 5



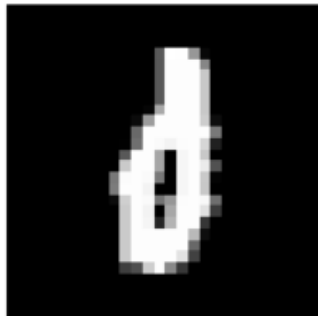
Real: 7, Pred: 4



Real: 6, Pred: 6

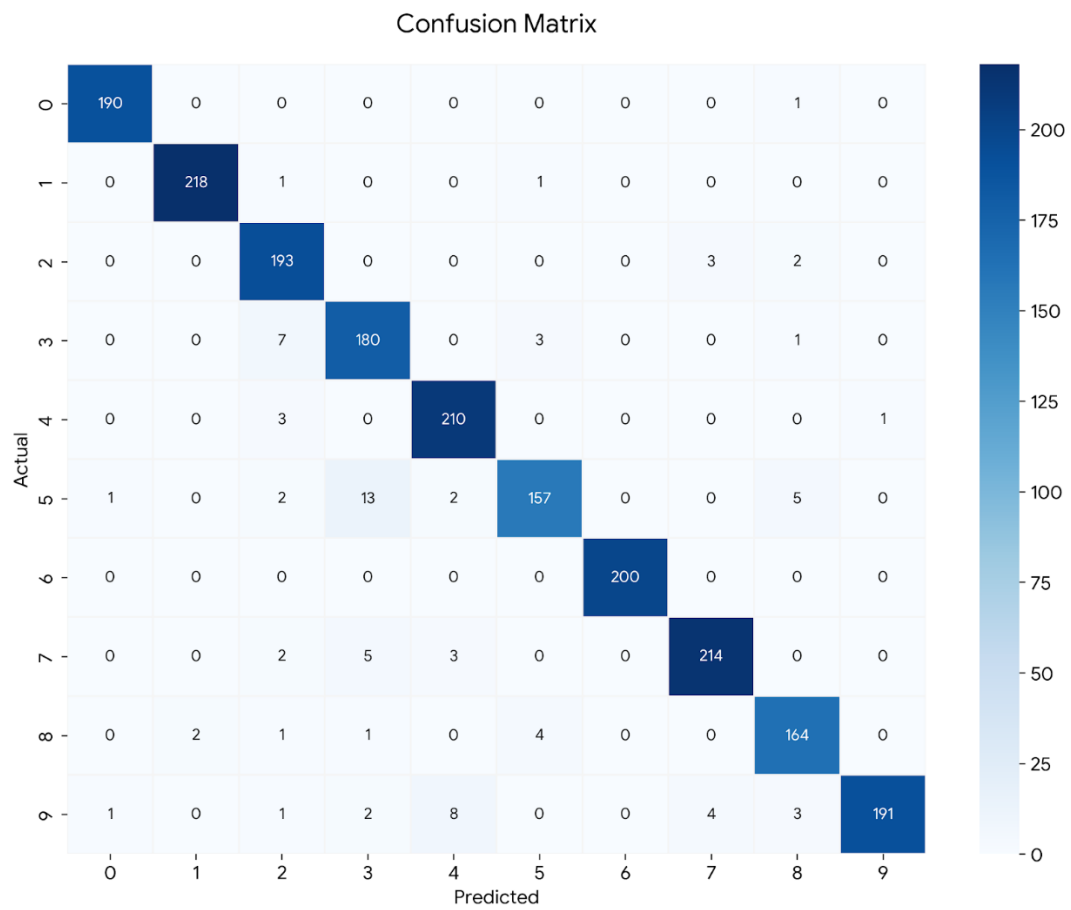


Real: 0, Pred: 8



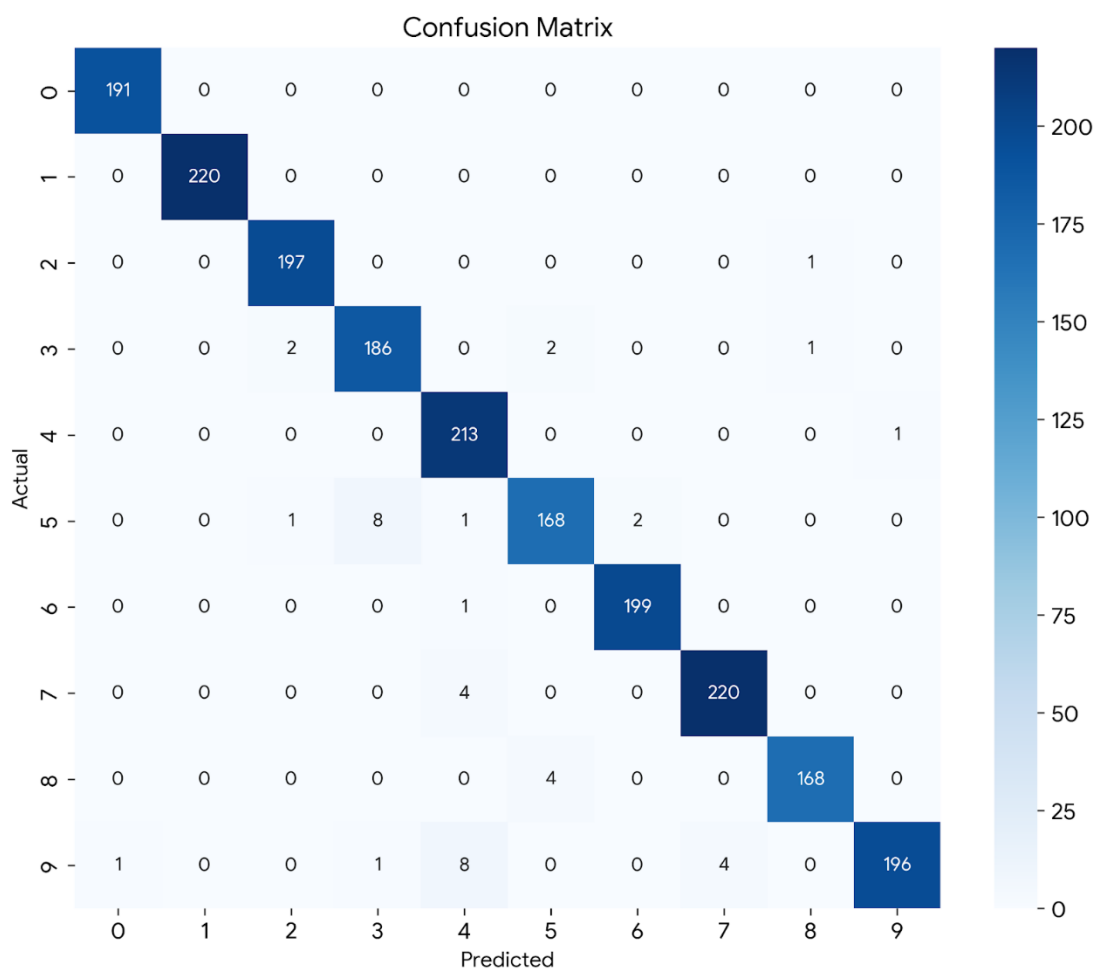
learning_rate = 0.01

epochs = 10



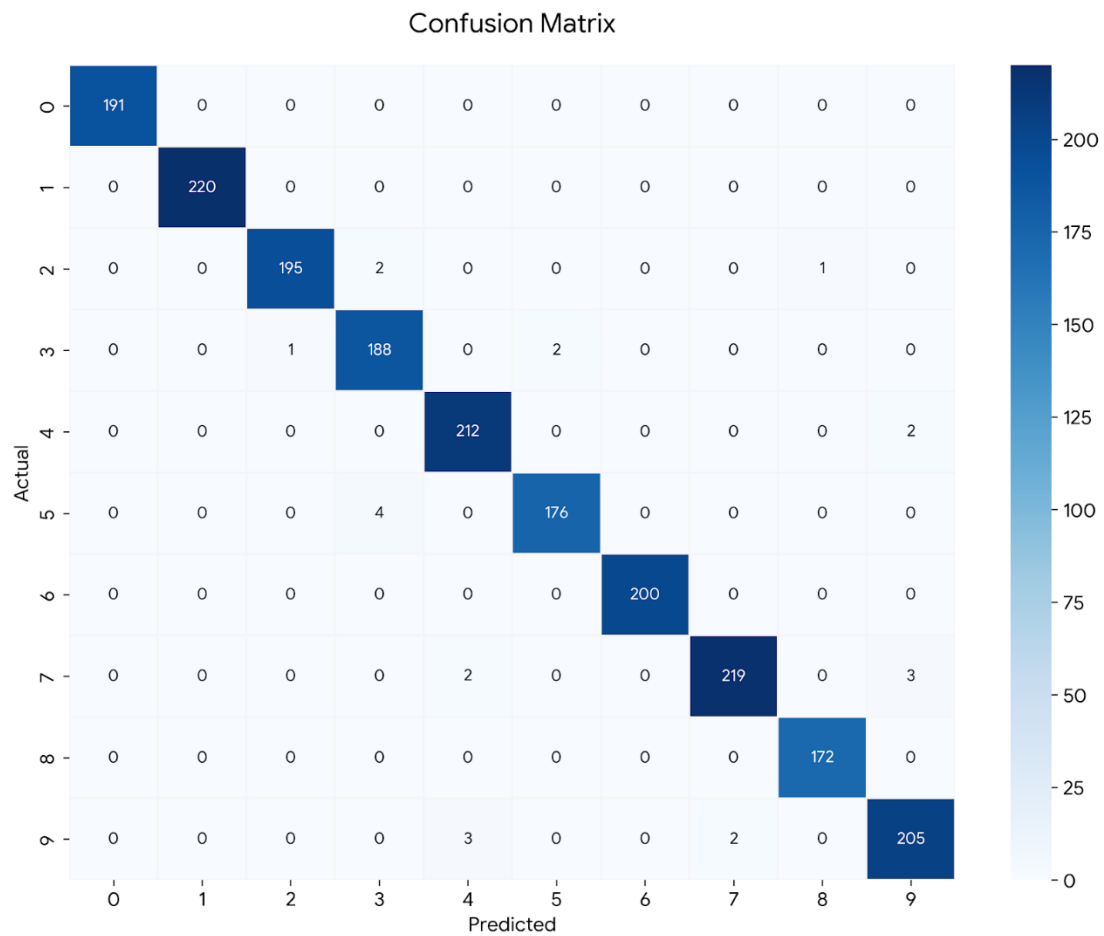
learning_rate = 0.001

epochs = 20



learning_rate = 0.0001

epochs = 30



Δεύτερη φάση 1 V 1

Σε αυτήν την φάση αποφάσισα να διαλέξω τον χαρακτήρα 8 για την εκπαίδευση με 1000 δείγματα και τα υπόλοιπα με 50. Αυτό το έκανα για να εμφανιστεί το φαινόμενο του Νταή, δηλαδή επειδή μέρη του 8 περιέχονται σε πολλά νούμερα όπως το 9, 6 και πολλά ακόμα. Αν θέλαμε να παραμείνει όσο περισσότερο αντικειμενική και unbiased η ταξινόμηση θα έπρεπε να επιλέξουμε ένα στοιχείο το οποίο έχει σχέση με όσο τον δυνατόν λιγότερα όπως το 1.

Συλλογή 1000 δειγμάτων ανα κατηγορία 0...9 αρα 1450 δειγματα

```

lista_me_10_trains = []

topiki_boithitiki_lista_kategorias = []

topiki_boithitiki_lista_kategorias_gia_8 = []

j = 0

while len(topiki_boithitiki_lista_kategorias_gia_8) < 1000:
    if 8 == labels_lista_train[j]:
        topiki_boithitiki_lista_kategorias_gia_8.append(X_train[j])
        j+=1

for i in range(10):

    if i != 8:
        topiki_boithitiki_lista_kategorias = []

        j = 0

        while len(topiki_boithitiki_lista_kategorias) < 50:
            if i == labels_lista_train[j]:
                topiki_boithitiki_lista_kategorias.append(X_train[j])
                j+=1
        lista_me_10_trains.append(topiki_boithitiki_lista_kategorias)
    else:
        lista_me_10_trains.append(topiki_boithitiki_lista_kategorias_gia_8)

print('lens',len(lista_me_10_trains))
for i in range(10):
    print(i, ' ',len(lista_me_10_trains[i]))

```

✓ 0.0s

```

lens 10
0 50
1 50
2 50
3 50
4 50
5 50
6 50
7 50
8 1000
9 50

```

Τα βάρη

```

weights = []

learning_rate = 1

epochs = 1

for i in range(45):
    topiki_boithitiki_barwn = np.zeros(784)

    a = comparison[i][0]
    b = comparison[i][1] #παίρνω τις 2 συγγραφείς

    for e in range(epochs):
        if a != 8 and b !=8:
            eikones_a = lista_me_10_trains[a]
            eikones_b = lista_me_10_trains[b] #παίρνω τις 1000 εικόνες

            combined_a_and_b = np.array(eikones_a + eikones_b) #τα κάνω combine

            combined_a_and_b_labels = np.array([1] * 50 + [-1] * 50) #βάζω σαν labels τα 1 (α) και τα -1 (β)

            anakatema = np.arange(100) #laxnoi για το πως να ανακατέψω τα 2

            np.random.shuffle(anakatema) #ανακατέβω το anakatema

            combined_a_and_b_anakatemena = combined_a_and_b[anakatema] #εφαρμόζω το anakatema στο combined_a_and_b

            combined_a_and_b_labels_anakatemena = combined_a_and_b_labels[anakatema]

            for j in range(100):
                sum = 0

                for z in range(784):
                    sum += combined_a_and_b_anakatemena[j][z] * topiki_boithitiki_barwn[z]

                if apofasi(sum) == -1 and combined_a_and_b_labels_anakatemena[j] == 1:
                    for z in range(784):
                        topiki_boithitiki_barwn[z] += combined_a_and_b_anakatemena[j][z] * learning_rate
                elif apofasi(sum) == 1 and combined_a_and_b_labels_anakatemena[j] == -1:
                    for z in range(784):
                        topiki_boithitiki_barwn[z] -= combined_a_and_b_anakatemena[j][z] * learning_rate

            # weights.append(topiki_boithitiki_barwn)

        else:

```

Ο αλγόριθμος υπολογισμού των βαρών παραμένει στην ίδια φιλοσοφία με πριν με εξαίρεση τα 1000 δείγματα του χαρακτήρα 8.

```

else:
    eikones_a = lista_me_10_trains[a]
    eikones_b = lista_me_10_trains[b] #παίρνω τις 1000 εικόνες

    combined_a_and_b = np.array(eikones_a + eikones_b) #τα κάνω combine
    if a == 8:
        combined_a_and_b_labels = np.array([1] * 1000 + [-1] * 50) #βάζω σαν labels τα
    else:
        combined_a_and_b_labels = np.array([1] * 50 + [-1] * 1000) #βάζω σαν labels τα

    anakatema = np.arange(1050) #laxnoi gia to πως να
    np.random.shuffle(anakatema) #ανακατέβω το anakatema
    combined_a_and_b_anakatemena = combined_a_and_b[anakatema] #εφαρμόζω το anakatema
    combined_a_and_b_labels_anakatemena = combined_a_and_b_labels[anakatema]

    for j in range(1050):
        sum = 0

        for z in range(784):
            sum += combined_a_and_b_anakatemena[j][z] * topiki_boithitiki_barwn[z]

        if apofasi(sum) == -1 and combined_a_and_b_labels_anakatemena[j] == 1:
            for z in range(784):
                topiki_boithitiki_barwn[z] += combined_a_and_b_anakatemena[j][z] * learning_rate
        elif apofasi(sum) == 1 and combined_a_and_b_labels_anakatemena[j] == -1:
            for z in range(784):
                topiki_boithitiki_barwn[z] -= combined_a_and_b_anakatemena[j][z] * learning_rate

    weights.append(topiki_boithitiki_barwn)

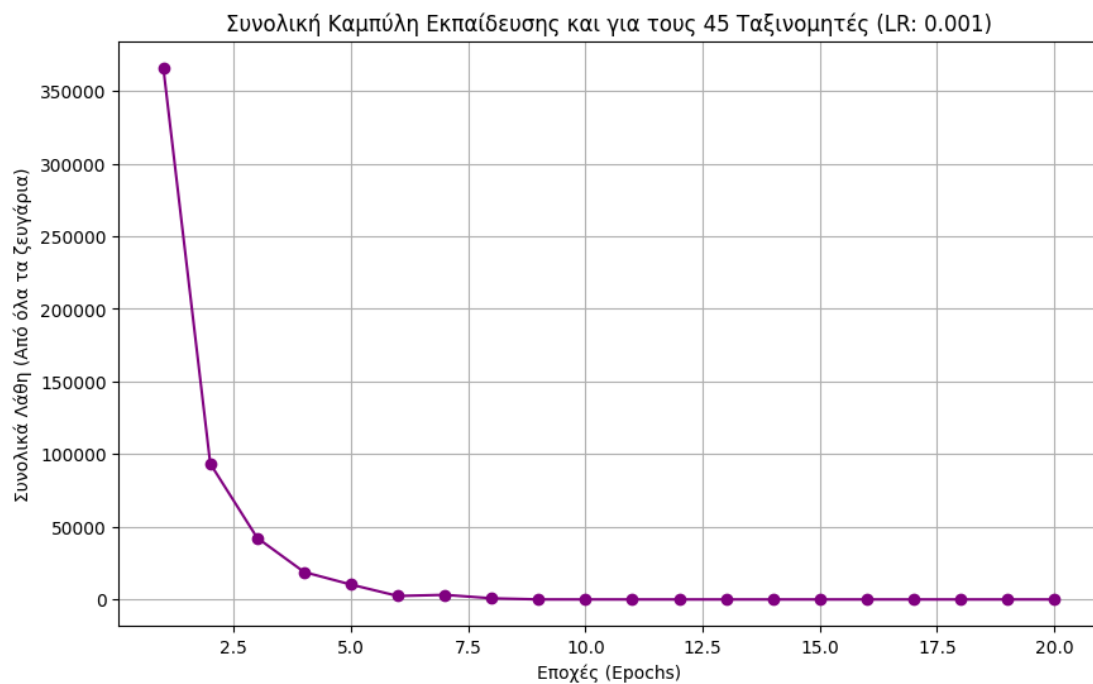
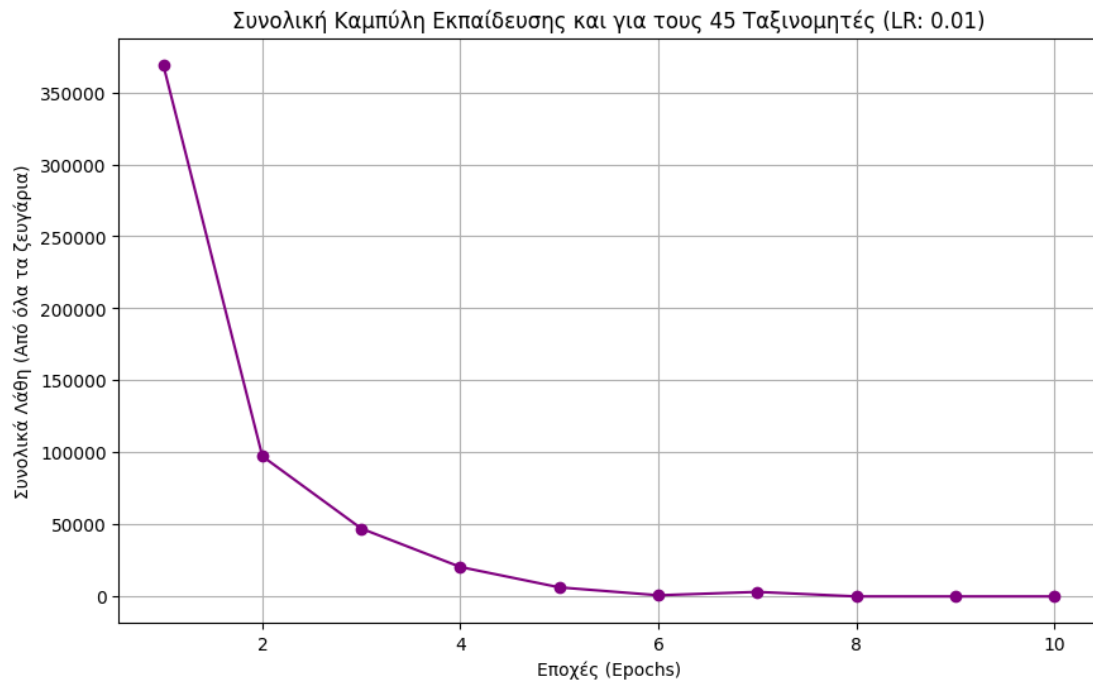
```

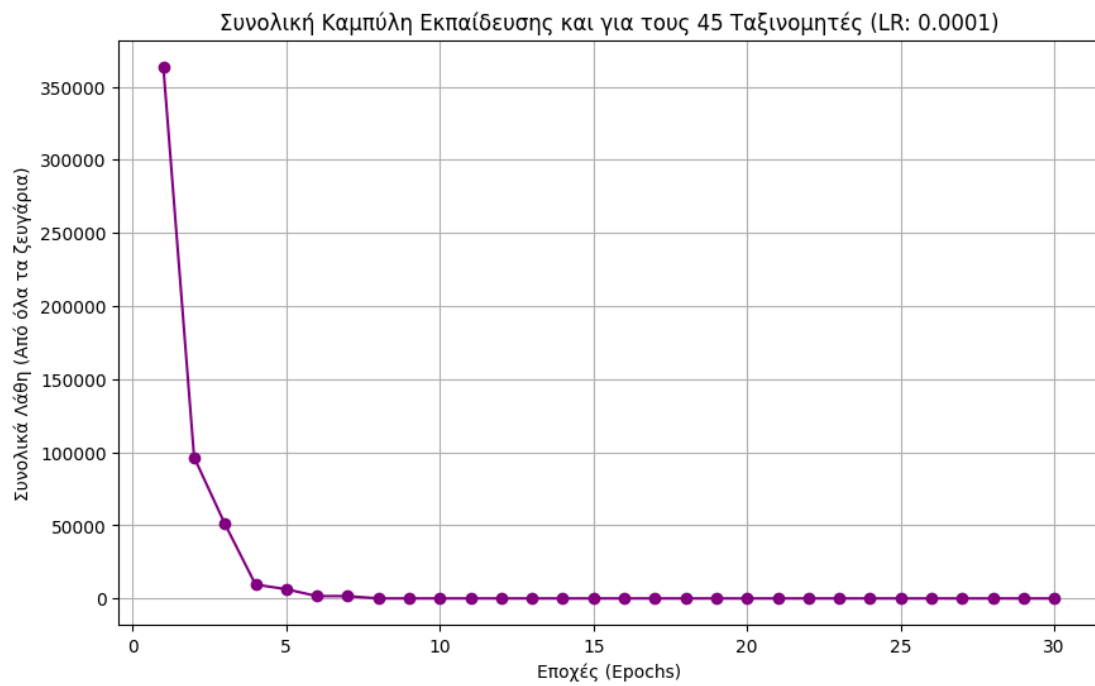
✓ 14.9s

Learning rate	Epochs	Επιτυχία στα 2000	Χρόνος
1	1	1347	4.3 sec
0.1	1	1448	4.4 sec
0.01	1	1387	4.3 sec
0.001	1	1348	4.3 sec
0.0001	1	1499	4.2 sec
1	10	1700	41.0 sec
1	20	1668	1min 26.2 sec
1	30	1633	2min 2.1 sec
0.1	10	1701	42.0 sec
0.1	20	1666	1min 27.7 sec
0.1	30	1671	2min 14.5 sec
0.01	10	1645	44.9 sec
0.01	20	1646	1min 27.9 sec
0.01	30	1649	2min 16.5 sec
0.0001	10	1637	44.1 sec
0.0001	20	1619	1 min 22.8 sec
0,0001	30	1650	2 min 11.0 sec

Σε σύγκριση με τα ίσα μεγέθη ανά κατηγορία βλέπουμε μεγάλη διαφορά στην ακρίβεια τους της τάξεις των 200.

Σε αντίθεση με την πρώτη φάση βλέπουμε στις καμπύλες πως η μείωση των σφαλμάτων έχει ολοκληρωθεί μετά τις 7 εποχές.



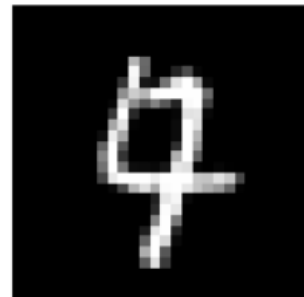


Παραδείγματα:

Real: 8, Pred: 8



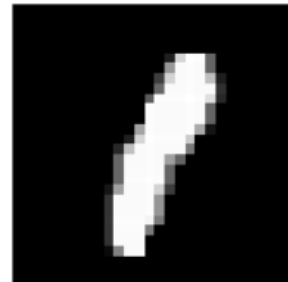
Real: 4, Pred: 7



Real: 2, Pred: 2



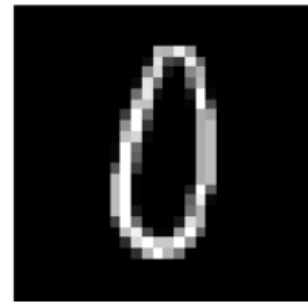
Real: 1, Pred: 8



Real: 0, Pred: 0

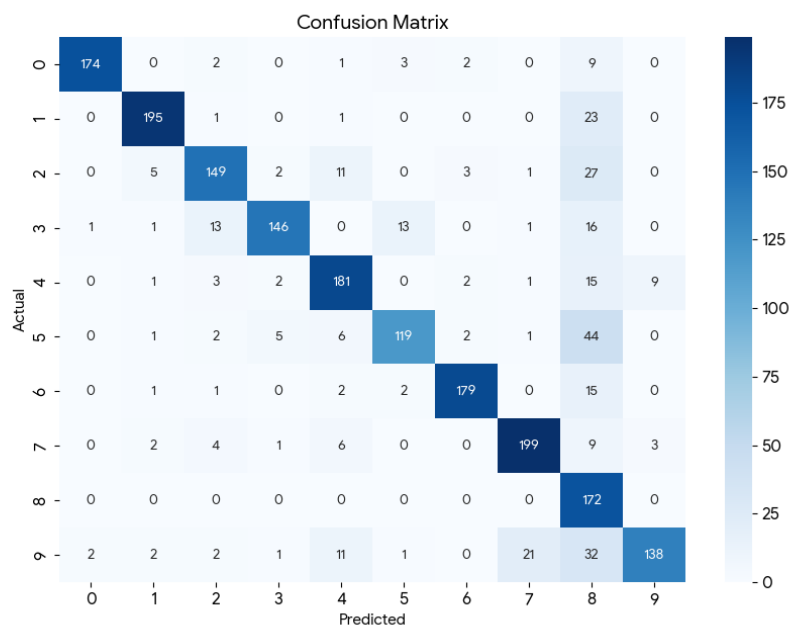


Real: 0, Pred: 5



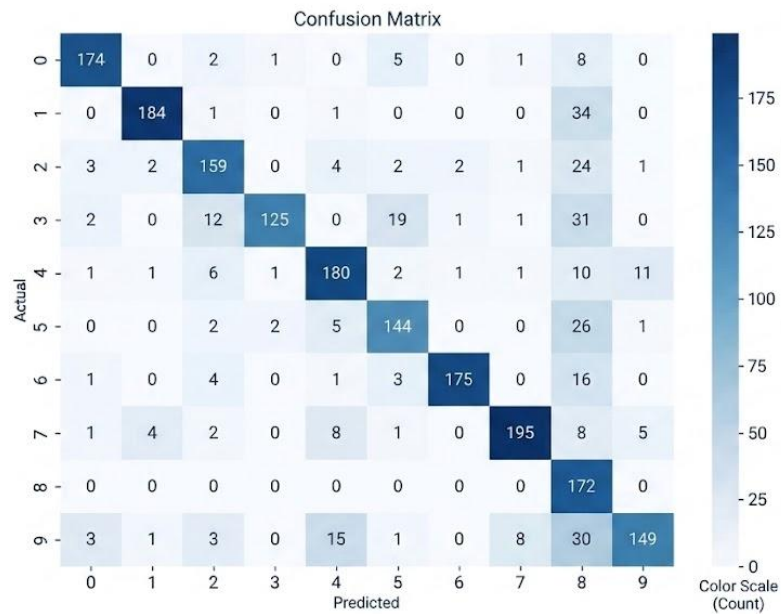
learning_rate = 0.01

epochs = 10



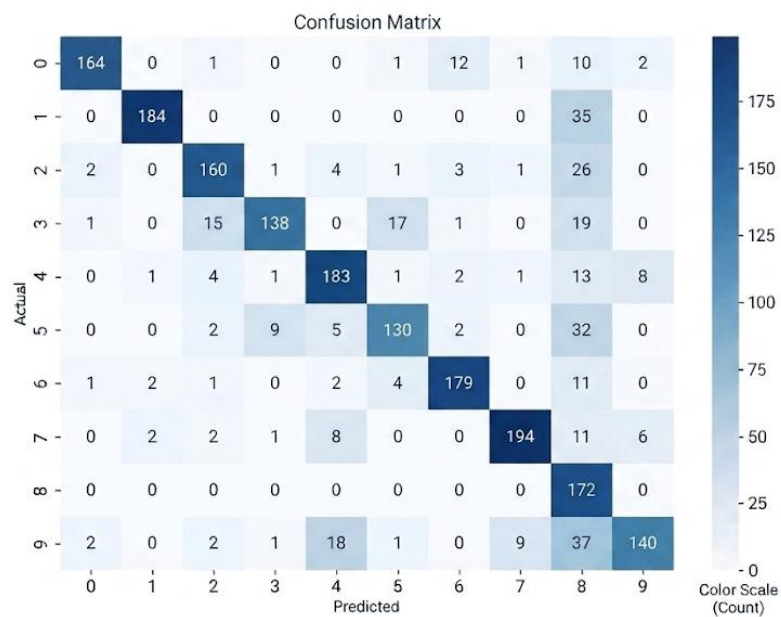
learning_rate = 0.001

epochs = 20



learning_rate = 0.0001

epochs = 30



Σύμφωνα με τους πίνακες υπάρχει ξεκάθαρη κλήση προς το 8 μιας και αυτό έχει πολλά περισσότερα δείγματα.

Πρώτη φάση Ο V A

```
weights = []
learning_rate = 1
epochs = 1

for sygkritis in range(10):

    topiki_boithitiki_barwn = np.zeros(784)

    for e in range(epochs):

        temp_labels_sygkrisewn = []

        for i in range(10):

            if i != sygkritis:
                temp_labels_sygkrisewn.extend([-1] * 1000)
            else:
                temp_labels_sygkrisewn.extend([1] * 1000)

        sygkriseis = np.array(lista_me_10_trains).reshape(10000, 784)

        # print(len(sygkriseis))

        labels_sygkrisewn = np.array(temp_labels_sygkrisewn)

        anakatema = np.arange(10000)

        np.random.shuffle(anakatema)

        sygkriseis_anakatemena = sygkriseis[anakatema]

        labels_sygkrisewn_anakatemena = labels_sygkrisewn[anakatema]

        for i in range(10000):

            sum = np.dot(sygkriseis_anakatemena[i], topiki_boithitiki_barwn)

            if apofasi(sum) == -1 and labels_sygkrisewn_anakatemena[i] == 1:

                topiki_boithitiki_barwn += sygkriseis_anakatemena[i] * \
                    learning_rate

            elif apofasi(sum) == 1 and labels_sygkrisewn_anakatemena[i] == -1:

                topiki_boithitiki_barwn -= sygkriseis_anakatemena[i] * \
                    learning_rate

        weights.append(topiki_boithitiki_barwn)
```

Για αυτό τον ταξινομητή χρειάστηκε η διαδικασία να επαναληφθεί 10 φορές και όχι 45 διότι πλέον φτιάχνουμε 10 ταξινομητές για κάθε αριθμό ενάντια προς όλους τους άλλους. Πάλι σε παρόμοια λογική με το One v One δημιουργούμε έναν πίνακα με τις σωστές τιμές (1) και έναν με τις λάθος (-1) για τα 10.000 δείγματα. Έπειτα ακολουθεί το shuffling για αποφυγή του overfitting και ο έλεγχος για αύξηση ή μείωση των pixels.

```
lista_gia_test = X_test[0:2000]

lista_gia_test_labels = labels_lista_test[0:2000]

sum_epityxiwn = 0

for deigma_test_idx in range(2000):

    apotelesma_arithmos = 0

    max_score_arithmos = -1

    for baros_idx in range(10):
        sum = np.dot(lista_gia_test[deigma_test_idx], weights[baros_idx])
        if sum > max_score_arithmos:
            max_score_arithmos = sum
            apotelesma_arithmos = baros_idx

    if lista_gia_test_labels[deigma_test_idx] == apotelesma_arithmos:
        sum_epityxiwn+=1

print(sum_epityxiwn)
```

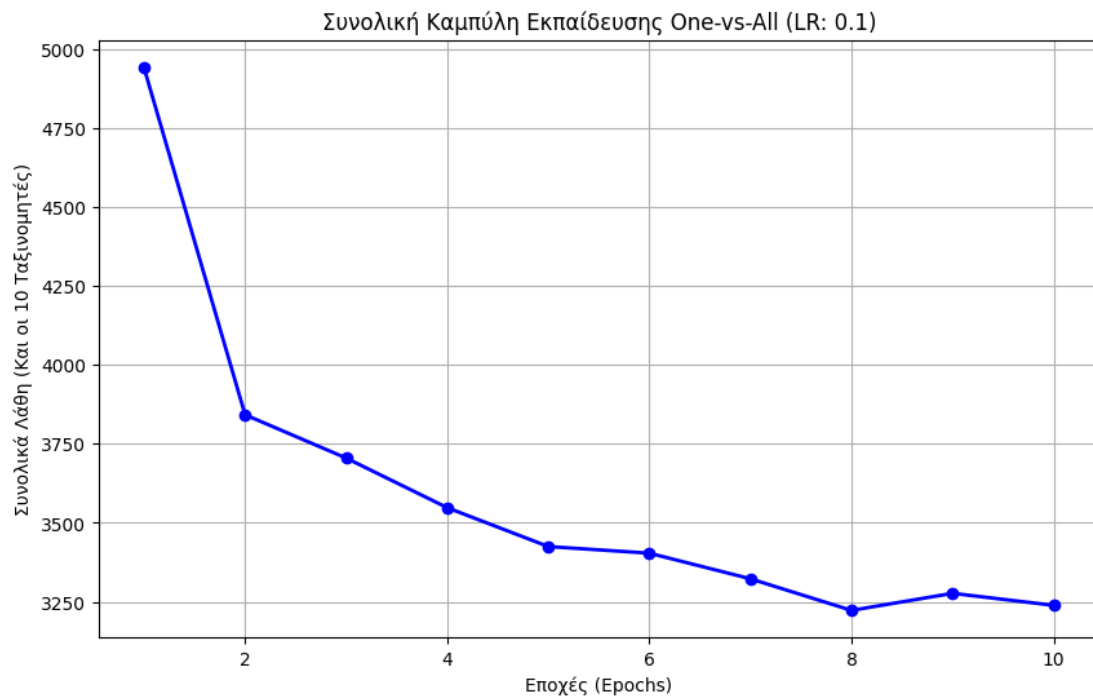
✓ 0.0s

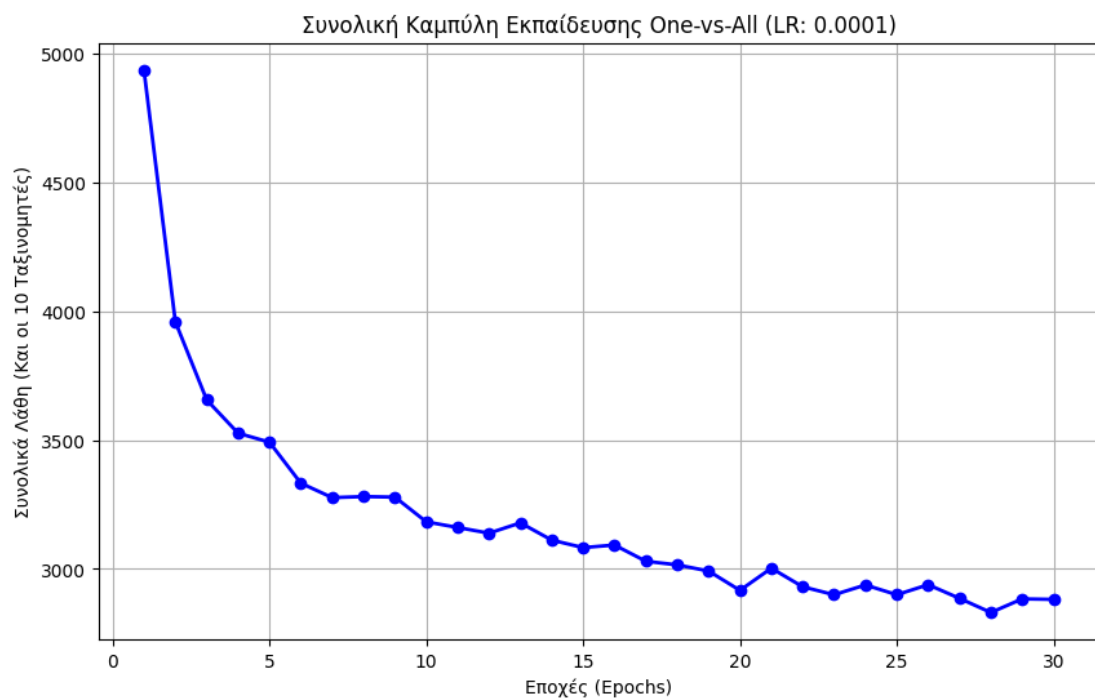
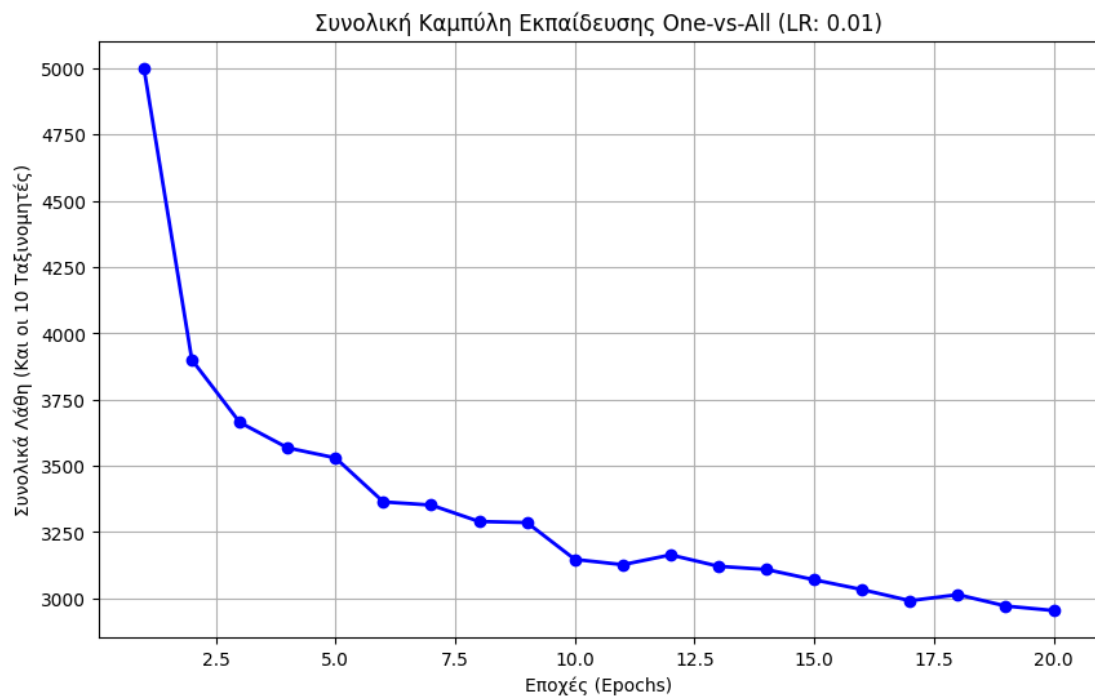
Το αποτέλεσμα της ταξινόμησης βγαίνει βάση ποιος θα έχει μεγαλύτερο αποτέλεσμα εσωτερικού γινομένου με το βάρος του,

Learning rate	Epochs	Επιτυχία στα 2000	Χρόνος
1	1	1738	0,5 sec
0.1	1	1746	0,5 sec
0.01	1	1713	0,5 sec
0.001	1	1674	0,4 sec
0.0001	1	1771	0,5 sec
1	10	1753	4.9 sec
1	20	1811	9,1 sec
1	30	1834	13,9 sec
0.1	10	1753	4,6 sec
0.1	20	1806	9 sec
0.1	30	1788	13,3 sec
0.01	10	1795	4,6 sec

0.01	20	1779	9.6 sec
0.01	30	1854	13,4 sec
0.0001	10	1778	4,7 sec
0.0001	20	1777	9,1 sec
0,0001	30	1818	14,4 sec

Όπως βλέπουμε τα αποτελέσματα έχουν χαμηλότερη ακρίβεια από τον αλγόριθμο One ν One αλλά είναι πιο γρήγορος.

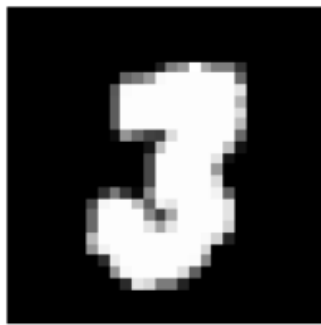




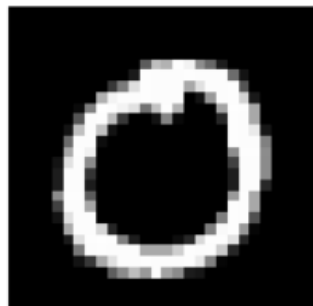
Real: 4, Pred: 4



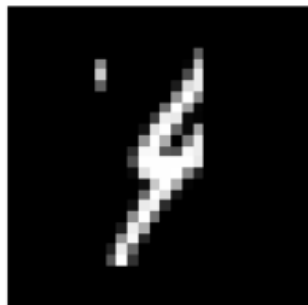
Real: 3, Pred: 1



Real: 0, Pred: 0



Real: 4, Pred: 1



Real: 8, Pred: 8

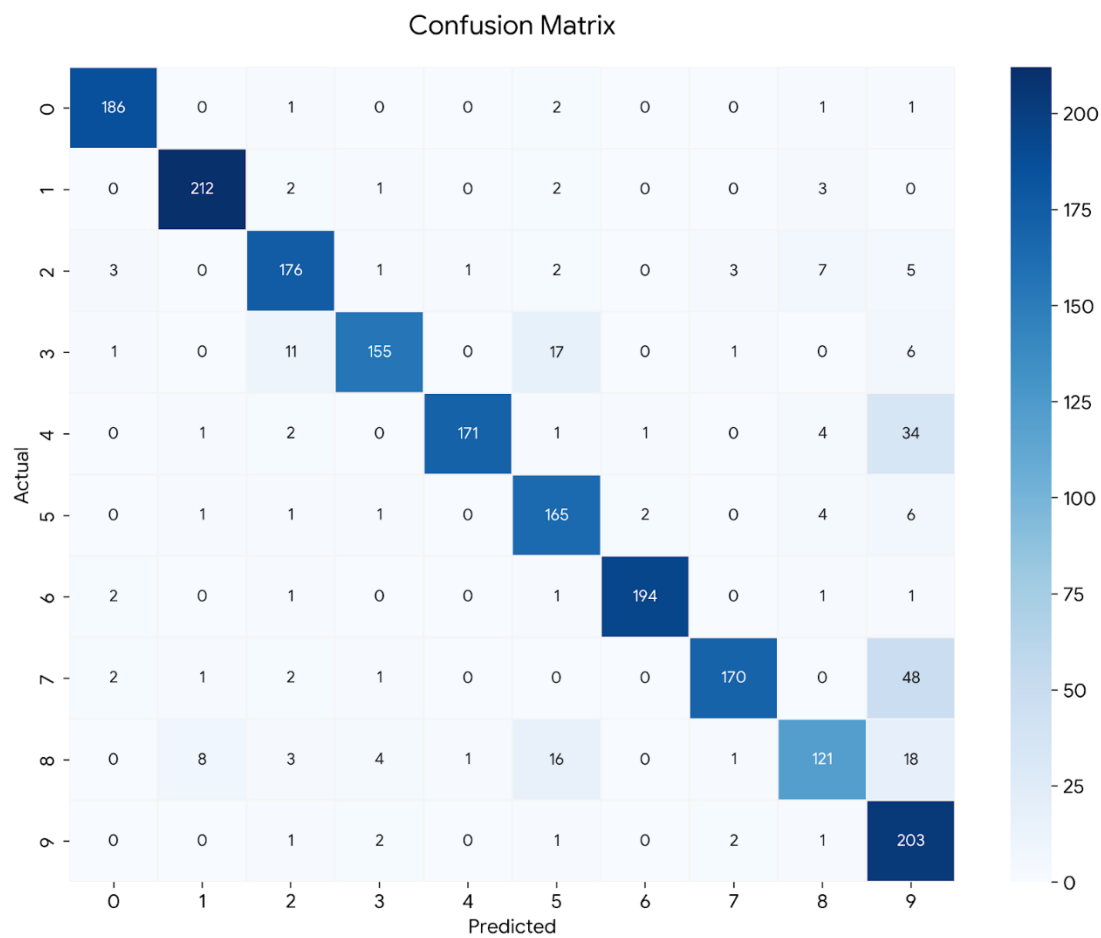


Real: 7, Pred: 4



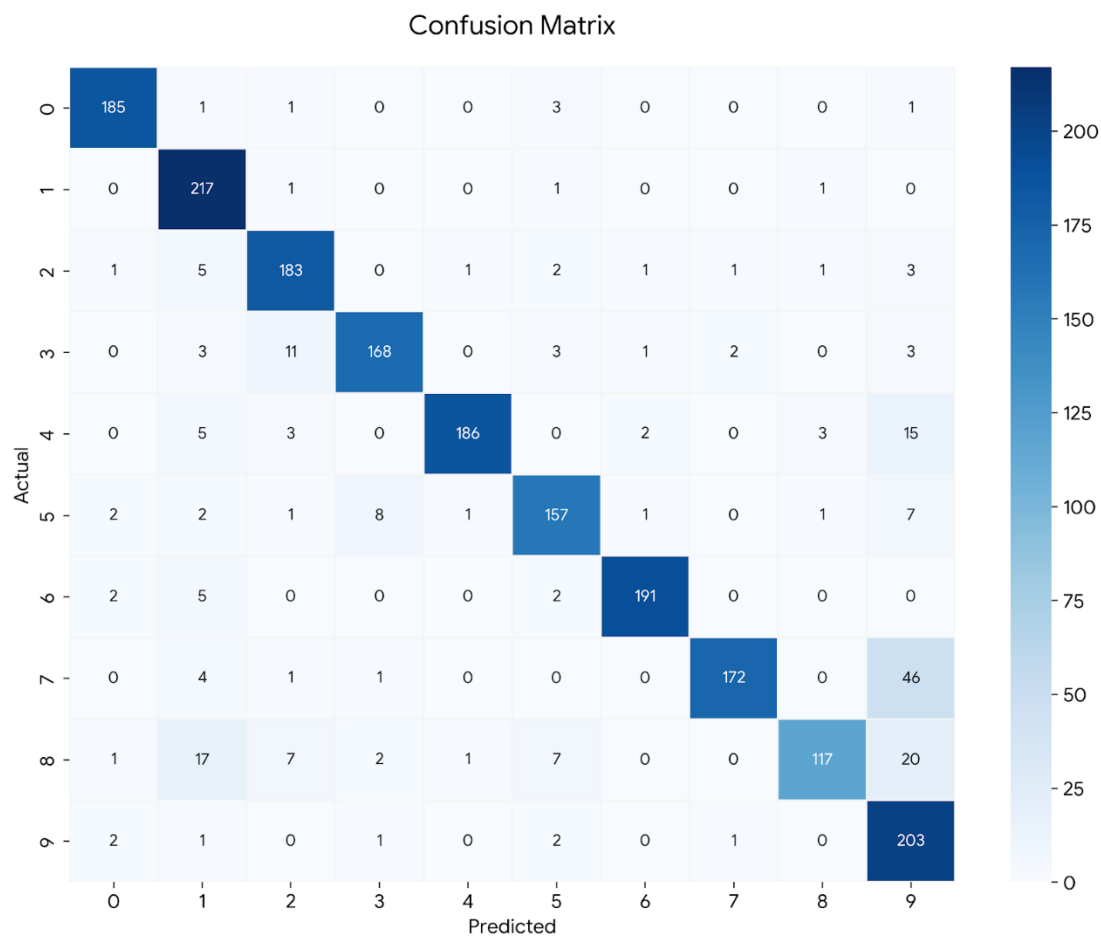
learning_rate = 0.1

epochs = 10



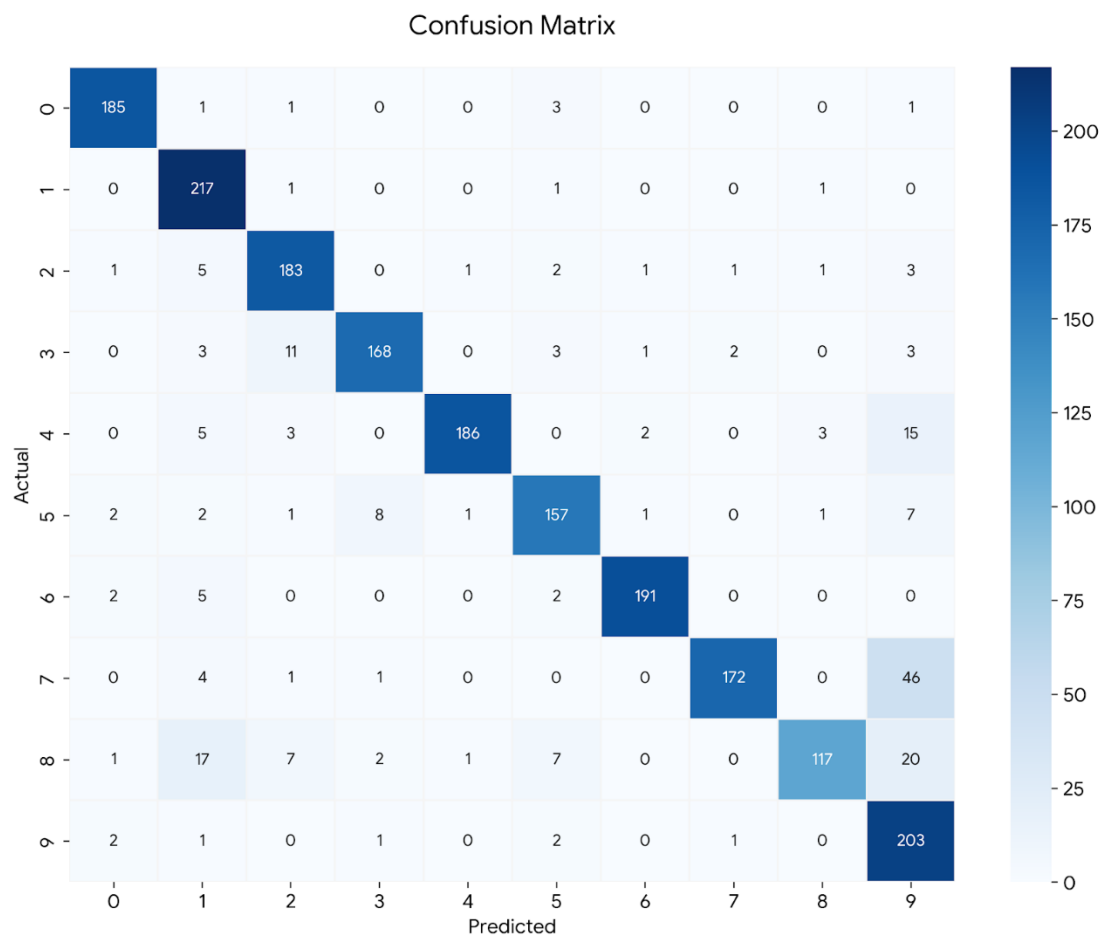
learning_rate = 0.01

epochs = 20



learning_rate = 0.0001

epochs = 30



Υπάρχει μεγάλη συσχέτιση του 9 με αριθμούς όπως το 7 ή το 4 αλλά αυτό είναι λογικό μιας και μοιάζουν αρκετά σαν σχήματα.

Δεύτερη φάση 1 V All

Για την δυσανάλογη παροχή δεδομένων επέλεξα το 8 και η υλοποίηση του γίνεται με ίδιο ρυθμό με την πρώτη φάση.

```

weights = []

learning_rate = 1

epochs = 10

istoria_lathwn_synolika = [0] * epochs

for sygkritis in range(10):
    topiki_boithitiki_barwn = np.zeros(784)

    istoria_lathwn = []

    for e in range(epochs):

        lathi_epoxis = 0

        temp_labels_sygkrisewn = []

        for i in range(10):
            if i == sygkritis and i == 8:
                temp_labels_sygkrisewn.extend([1] * 1000)
            elif i != sygkritis and i == 8:
                temp_labels_sygkrisewn.extend([-1] * 1000)
            elif i != sygkritis:
                temp_labels_sygkrisewn.extend([-1] * 50)
            else:
                temp_labels_sygkrisewn.extend([1] * 50)

        topiki_boithitiki_sygkrisewn = []

        for i in range(10):
            topiki_boithitiki_sygkrisewn.extend(lista_me_10_trains[i])

        sygkriseis = np.array(topiki_boithitiki_sygkrisewn)

        # print(len(sygkriseis))

        labels_sygkrisewn = np.array(temp_labels_sygkrisewn)

        anakatema = np.arange(1450)

        np.random.shuffle(anakatema)

        sygkriseis_anakatemena = sygkriseis[anakatema]

        labels_sygkrisewn_anakatemena = labels_sygkrisewn[anakatema]

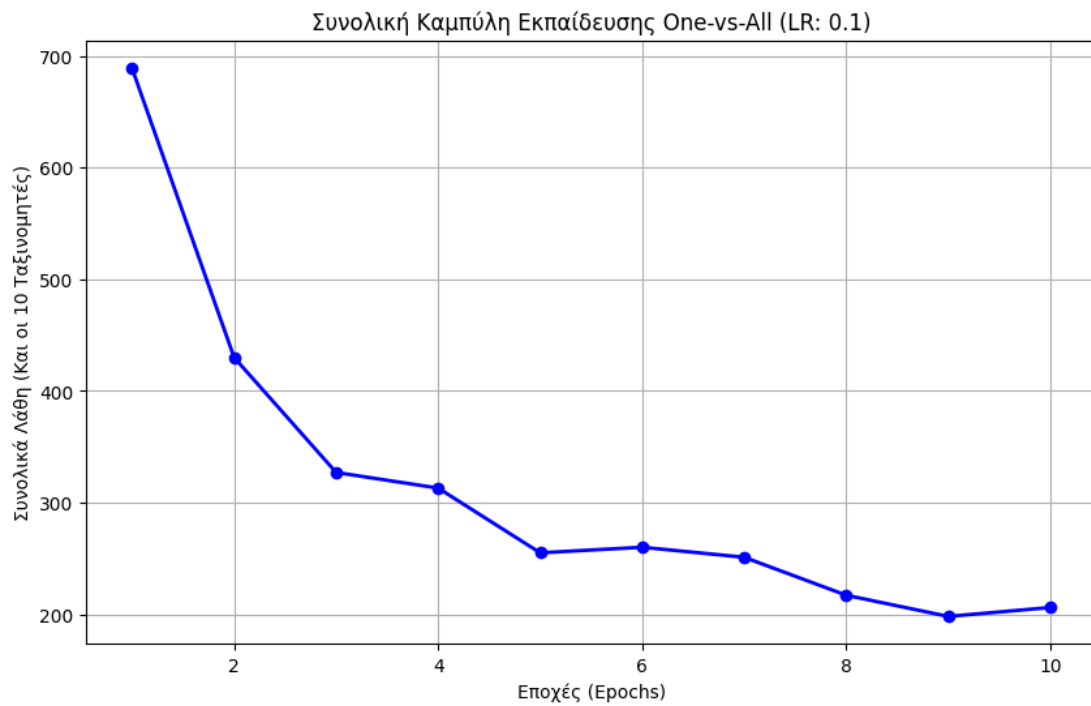
        for i in range(1450):

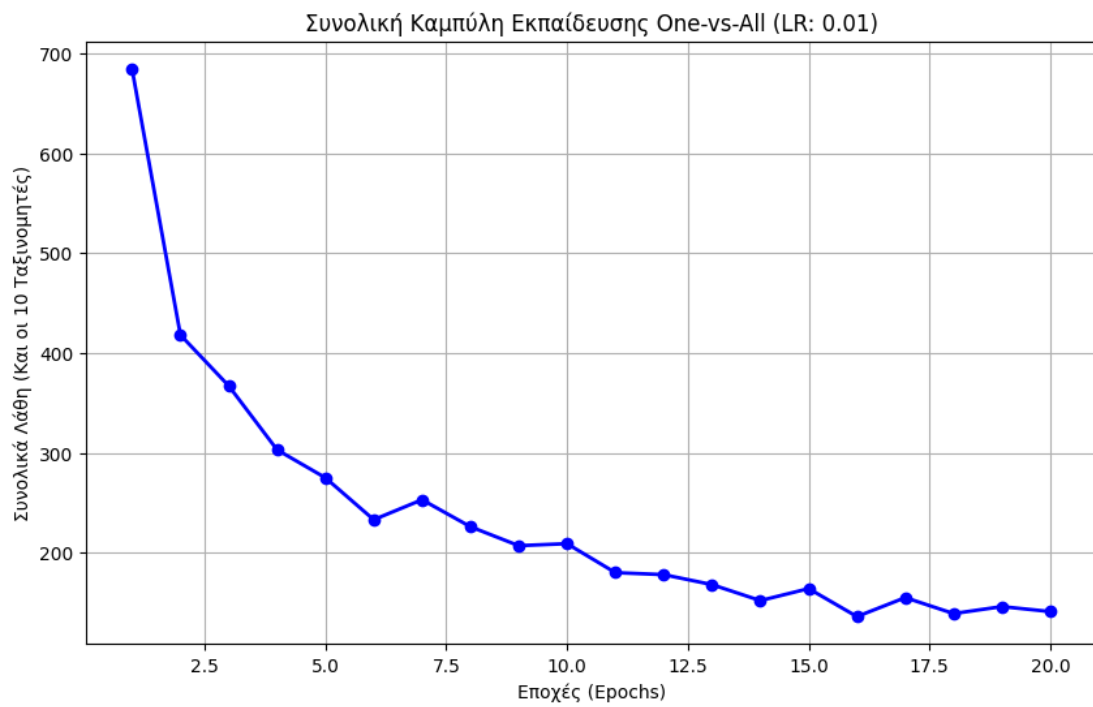
```

Learning rate	Epochs	Επιτυχια στα 2000	Χρόνος
1	1	1313	0.5
0.1	1	979	0.5
0.01	1	1379	0.1
0.001	1	1309	0.1
0.0001	1	1183	0.1
1	10	1378	1.1
1	20	1576	1
1	30	1473	1.7
0.1	10	1512	0,9
0.1	20	1486	1,1
0.1	30	1466	1,6
0.01	10	1557	0,5
0.01	20	1590	1,7

0.01	30	1637	1,7
0.0001	10	1412	0,5
0.0001	20	1403	1,4
0,0001	30	1369	2.1

Σε αυτή την περίπτωση η απώλεια είναι πολύ μεγάλη και η ακρίβεια έχει πέσει κατά πολύ.





Real: 9, Pred: 9



Real: 3, Pred: 8



Real: 5, Pred: 5



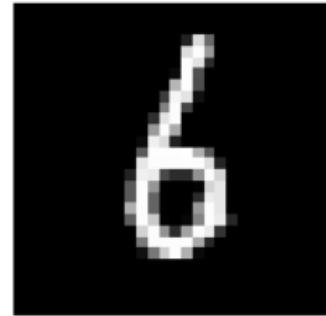
Real: 7, Pred: 8



Real: 8, Pred: 8



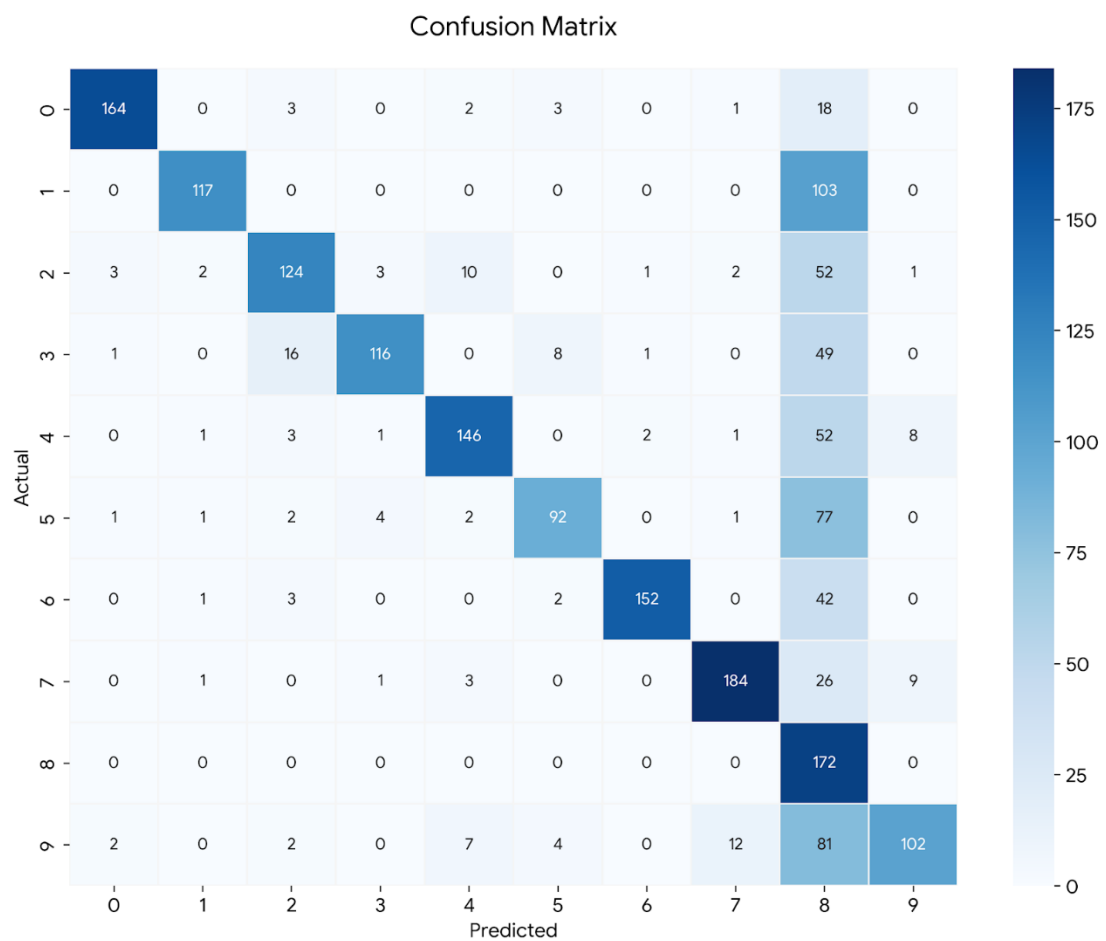
Real: 6, Pred: 8



Τα παραδείγματα λάθους ταξινόμησης είναι biased προς το 8 το οποίο είναι λογικό, ακόμα και αν ο γραφικός χαρακτήρας είναι ξεκάθαρος.

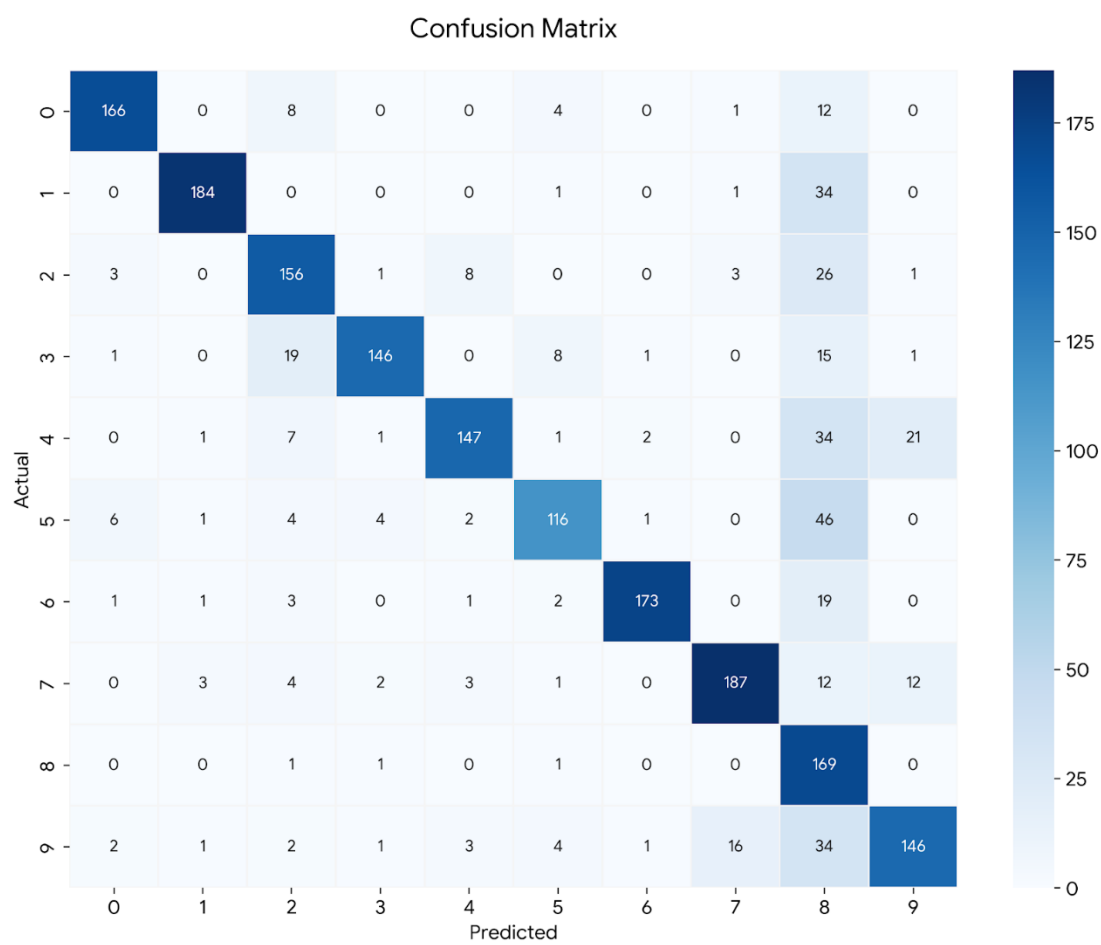
learning_rate = 0.1

epochs = 10



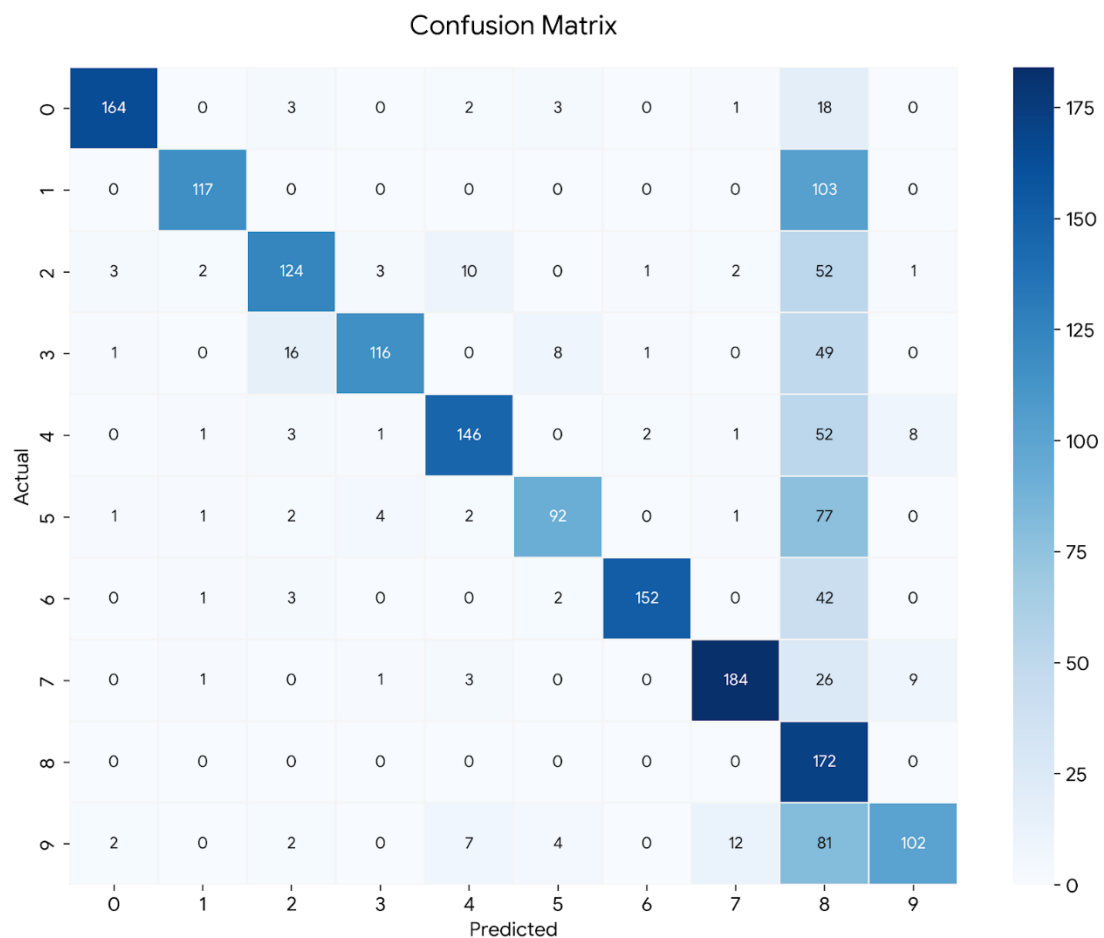
learning_rate = 0.01

epochs = 20



learning_rate = 0.0001

epochs = 30



Πρώτη φάση softmax

Σε αυτό τον αλγόριθμο χρειάστηκε να φτιάξω έναν ενιαίο ταξινομητή και για τα 10 νούμερα. Αρχικά δημιούργησα έναν πίνακα με τις σωστές τιμές και αντιστοιχίες στις προβλέψεις των αριθμών. Στην συνέχεια έκανα flattening στους πίνακες δεδομένων και στα labels τους και τα ανακάτεψα.

Η διαδικασία εκπαίδευσης του ταξινομητή Softmax πραγματοποιείται επαναληπτικά για κάθε δείγμα του συνόλου εκπαίδευσης και χωρίζεται σε τρία βασικά στάδια:

1. Εμπρόσθιο Πέρασμα (Forward Pass): Αρχικά, υπολογίζεται το εσωτερικό γινόμενο (np.dot) μεταξύ του πίνακα των βαρών και του διανύσματος της εικόνας εισόδου. Το αποτέλεσμα είναι ένα διάνυσμα δέκα βαθμολογιών, όπου η καθεμία αντιστοιχεί σε μία από τις 10 πιθανές κλάσεις (ψηφία 0-9).

2. Κανονικοποίηση και Υπολογισμός Σφάλματος: Οι βαθμολογίες αυτές περνούν μέσα από τη συνάρτηση ενεργοποίησης Softmax. Στη συνέχεια, παίρνω την πραγματική κλάση του δείγματος (πίνακας alithia) και υπολογίζεται το διάνυσμα του σφάλματος. Το σφάλμα προκύπτει από την αφαίρεση της πραγματικής αλήθειας από τις προβλεπόμενες πιθανότητες.
3. Ενημέρωση Βαρών (Backward Pass): Με βάση το υπολογισμένο σφάλμα, εφαρμόζεται ο κανόνας ανανέωσης της Κάθοδου Κλίσης (Gradient Descent). Ολόκληρος ο πίνακας βαρών ενημερώνεται ταυτόχρονα, αφαιρώντας το εξωτερικό γινόμενο του διανύσματος σφάλματος με το διάνυσμα της εικόνας, πολλαπλασιασμένο με τον επιλεγμένο ρυθμό μάθησης.

```
learning_rate = 0.1

epochs = 10

alithia = []

for i in range(10):
    topiki_boithitiki_alithias = []
    for j in range(10):
        if i == j:
            topiki_boithitiki_alithias.append(1)
        else:
            topiki_boithitiki_alithias.append(0)
    alithia.append(topiki_boithitiki_alithias)

alithia = np.array(alithia)

weights = np.zeros(7840).reshape(10, 784)  #(10, 784)

istoria_lathwn_synolika = [0] * epochs

for e in range(epochs):
    arxiko_train_set_flat = np.array(lista_me_10_trains).reshape(10000,784)

    arxiko_labels_flat = np.array(lista_me_10_trains_labels)

    anakatema = np.arange(10000)

    np.random.shuffle(anakatema)

    train_set_flat = arxiko_train_set_flat[anakatema]

    labels_flat = arxiko_labels_flat[anakatema]

    lathi_epoxis = 0

    for deigma_train_idx in range(10000):

        apotelesma_weight = np.dot(weights, train_set_flat[deigma_train_idx])

        trexwn_alitheia = alithia[labels_flat[deigma_train_idx]]

        apotelesma_weight = softmax(apotelesma_weight)

        errors = apotelesma_weight - trexwn_alitheia

        weights = weights - learning_rate * np.outer(errors, train_set_flat[deigma_train_idx])

        if np.argmax(apotelesma_weight) != labels_flat[deigma_train_idx]:
            lathi_epoxis += 1

    istoria_lathwn_synolika[e] += lathi_epoxis
```

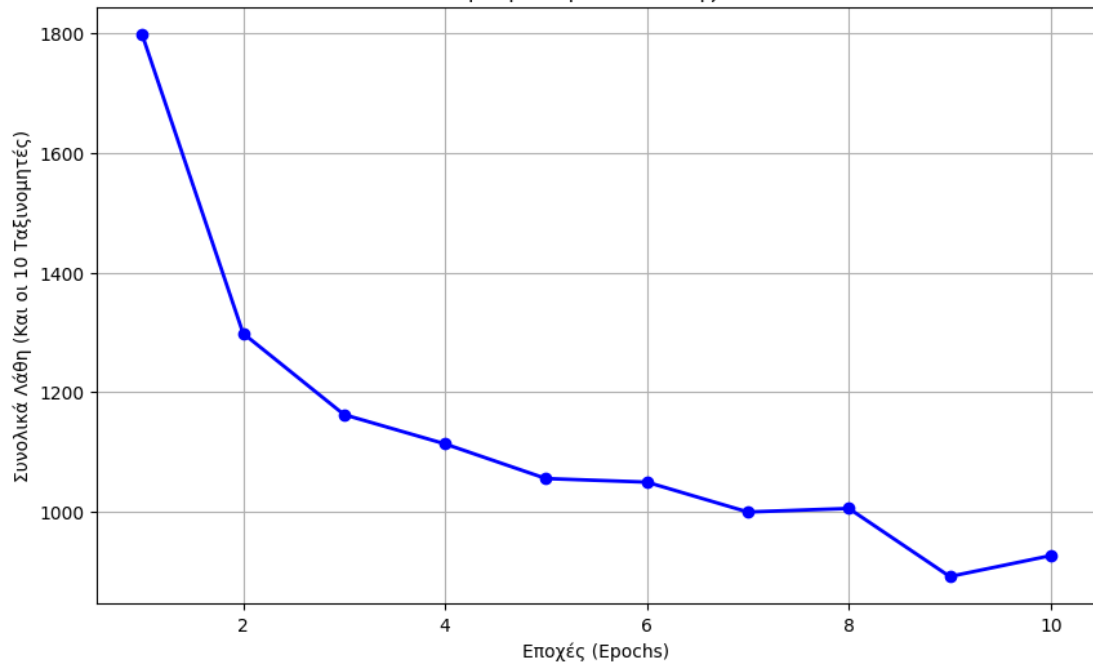
Εδώ υλοποιείται η συνάρτηση της softmax δηλαδή:

$$\frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$$

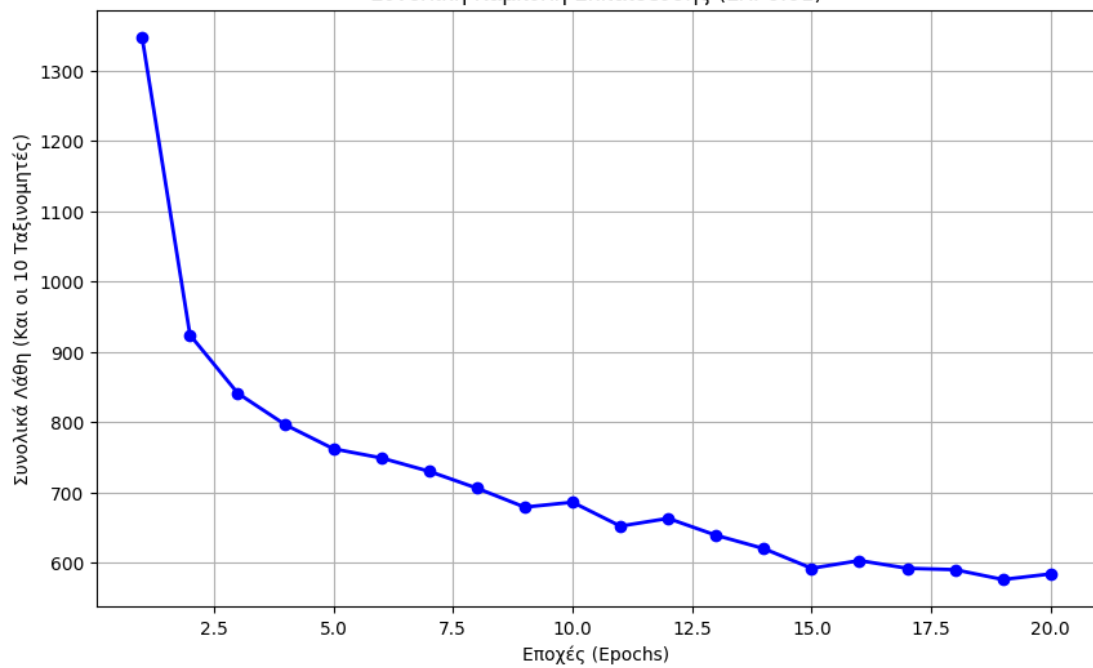
```
def softmax(x):  
    e = np.exp(x - np.max(x))  
    return e / e.sum()  
✓ 0.0s
```

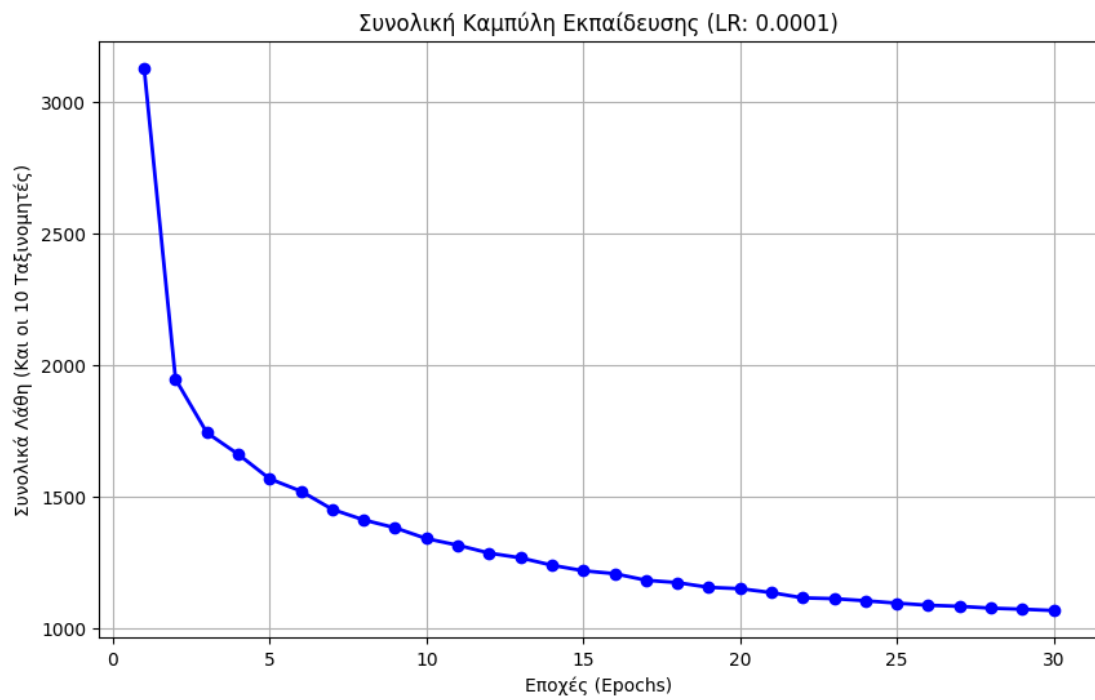
```
lista_gia_test = X_test[0:2000]  
  
lista_gia_test_labels = labels_lista_test[0:2000]  
  
sum_epityxiwn = 0  
  
for i in range(2000):  
    scores = np.dot(weights, lista_gia_test[i])  
    apotel = np.argmax(scores)  
    if apotel == lista_gia_test_labels[i]:  
        sum_epityxiwn +=1  
  
print(sum_epityxiwn)  
✓ 0.0s
```

Συνολική Καμπύλη Εκπαίδευσης (LR: 0.1)



Συνολική Καμπύλη Εκπαίδευσης (LR: 0.01)

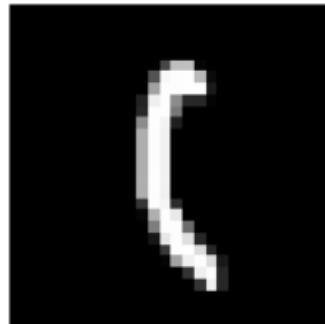




Real: 3, Pred: 3



Real: 1, Pred: 8



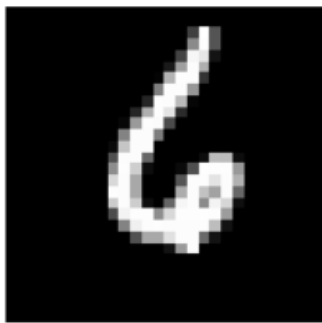
Real: 7, Pred: 7



Real: 3, Pred: 7



Real: 6, Pred: 6

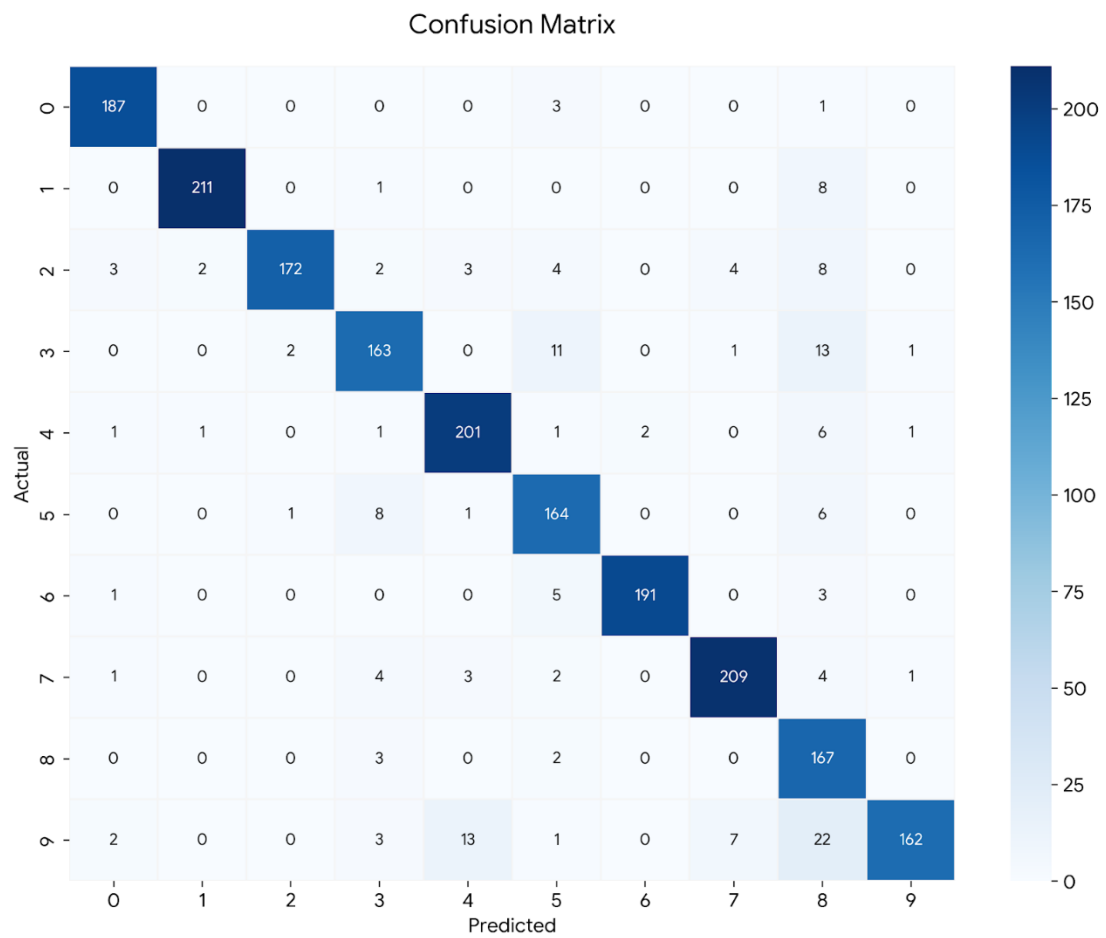


Real: 4, Pred: 8



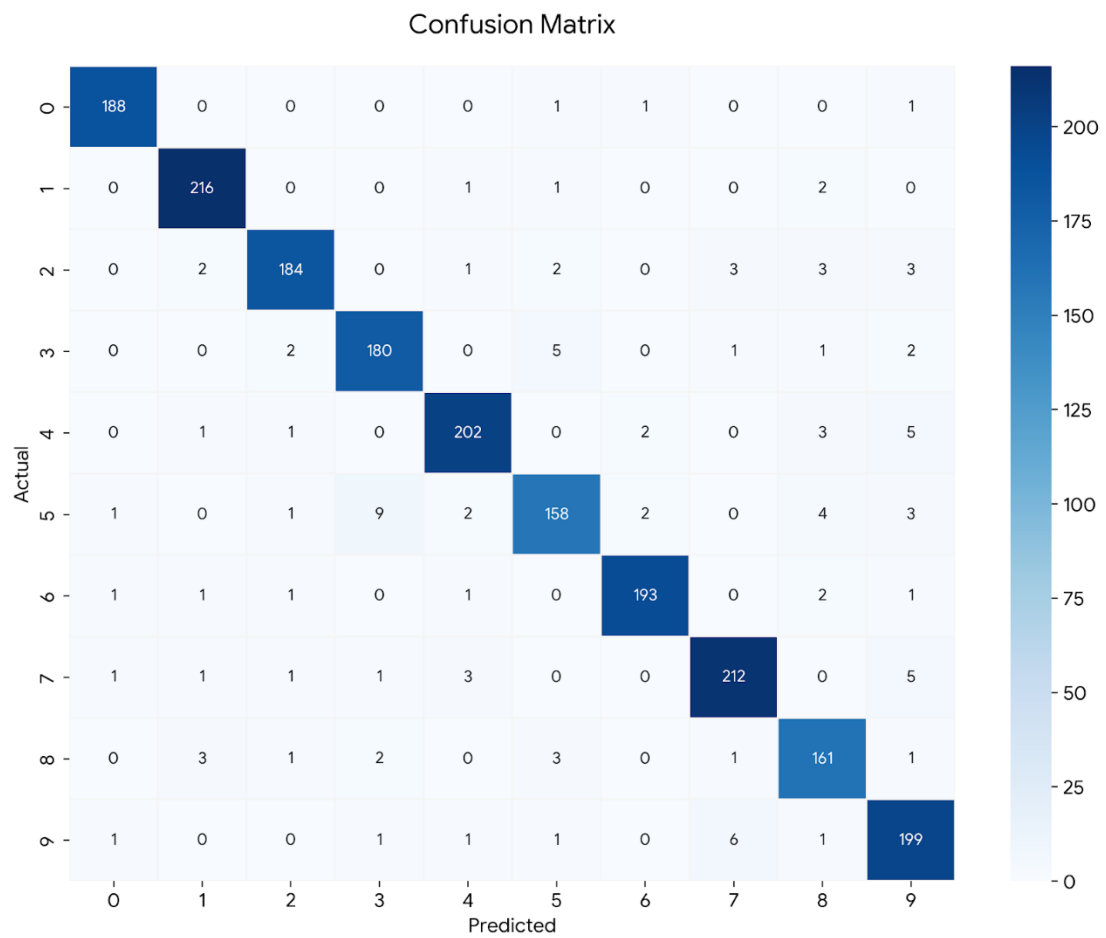
learning_rate = 0.1

epochs = 10



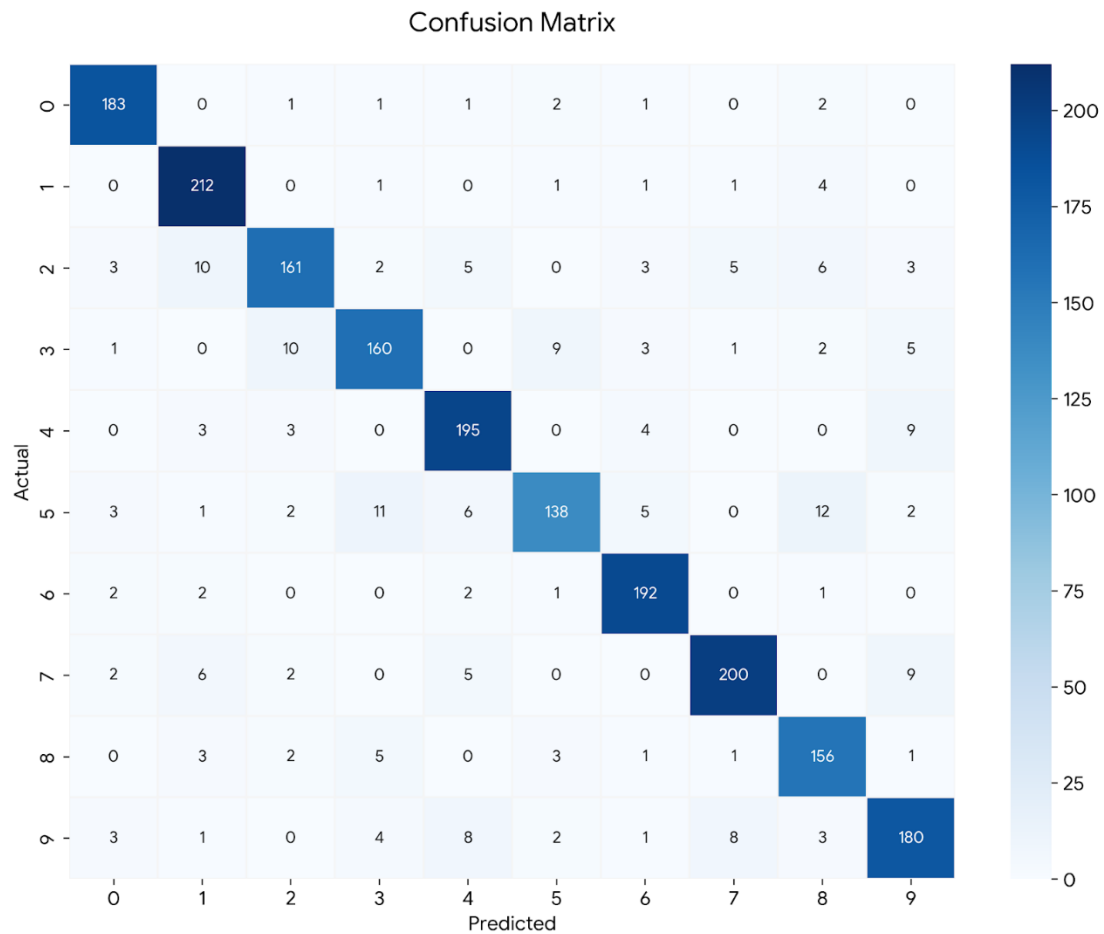
learning_rate = 0.001

epochs = 20



learning_rate = 0.0001

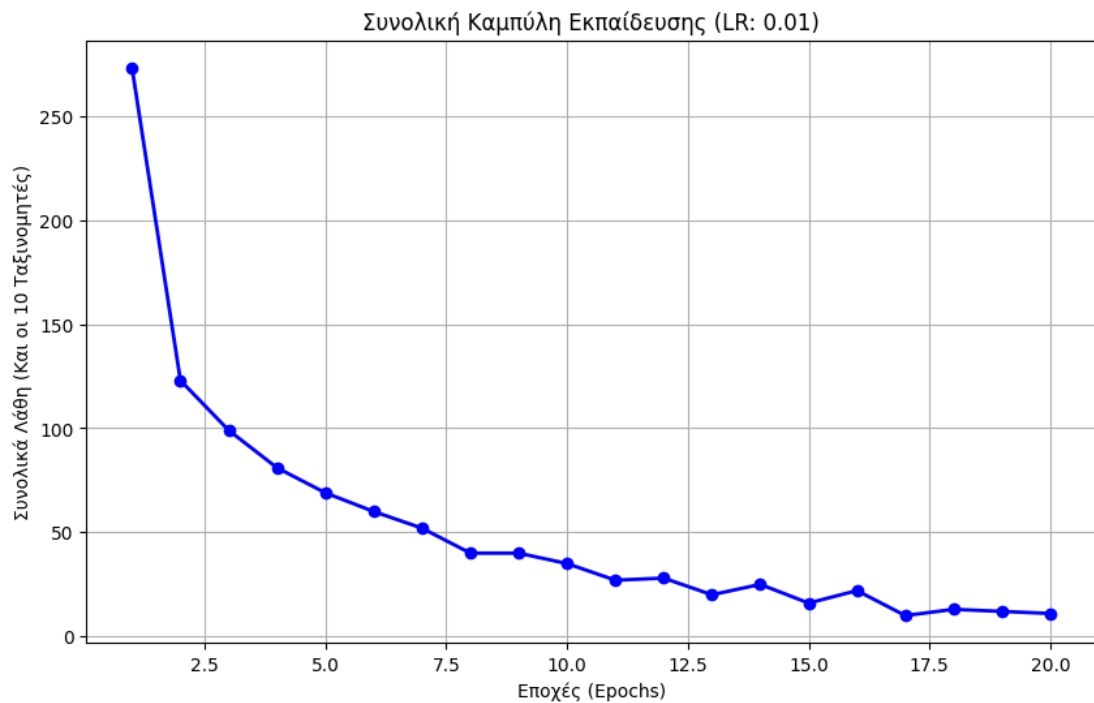
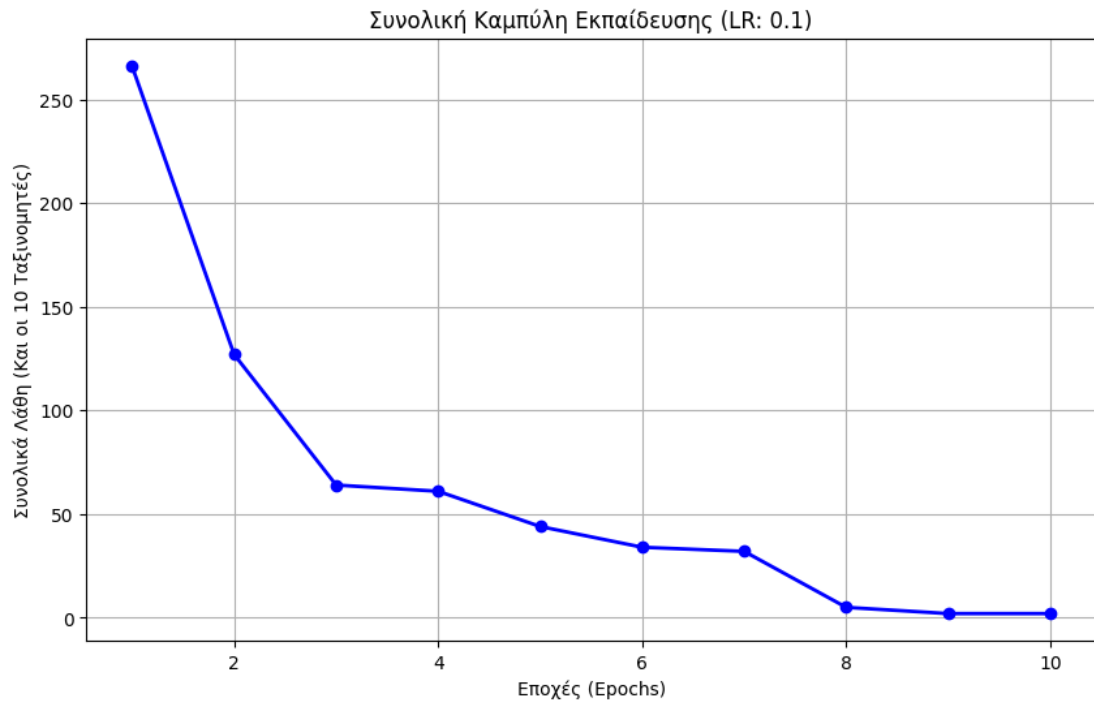
epochs = 30

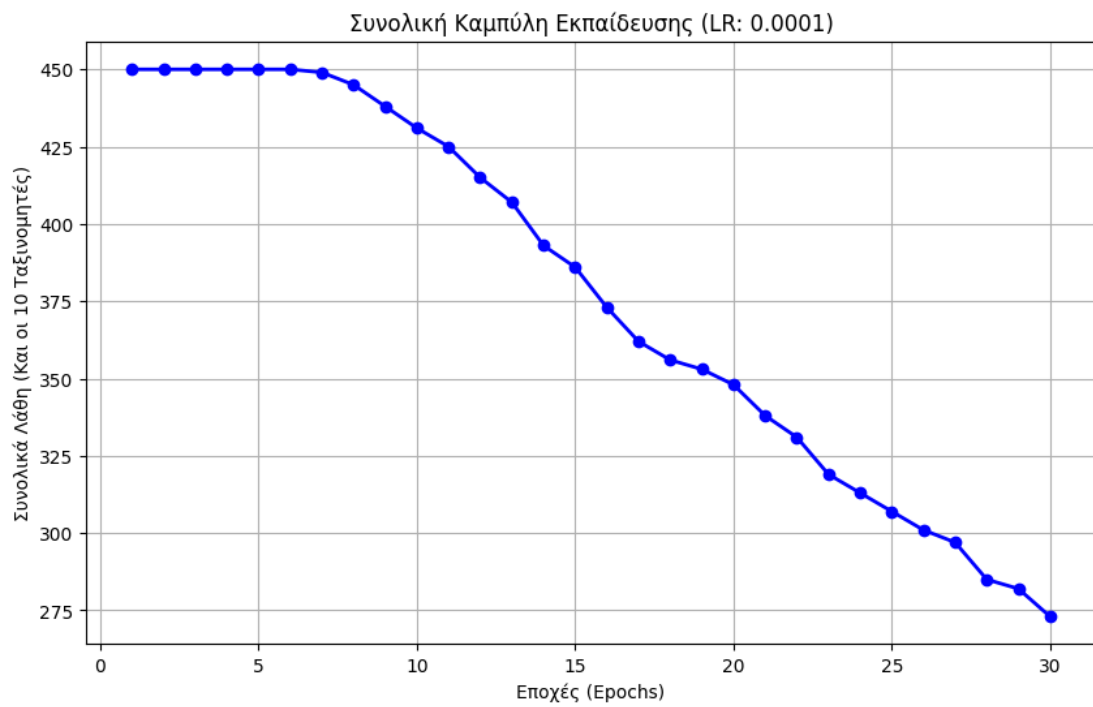


Δεύτερη φάση softmax

Learning rate	Epochs	Επιτυχία στα 2000	Χρόνος
1	1	1546	0,01
0.1	1	1419	0,01
0.01	1	1230	0,01
0.001	1	244	0,01
0.0001	1	172	0,01
1	10	1564	0,4
1	20	1676	1
1	30	1633	1,6
0.1	10	1619	0,4
0.1	20	1640	0,9
0.1	30	1630	1,4
0.01	10	1569	0,5
0.01	20	1614	0,9
0.01	30	1656	1,4

0.0001	10	240	0,4
0.0001	20	555	1
0,0001	30	794	1,4





Real: 4, Pred: 4



Real: 9, Pred: 8



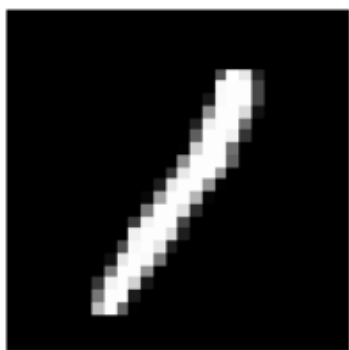
Real: 5, Pred: 5



Real: 2, Pred: 8



Real: 1, Pred: 1



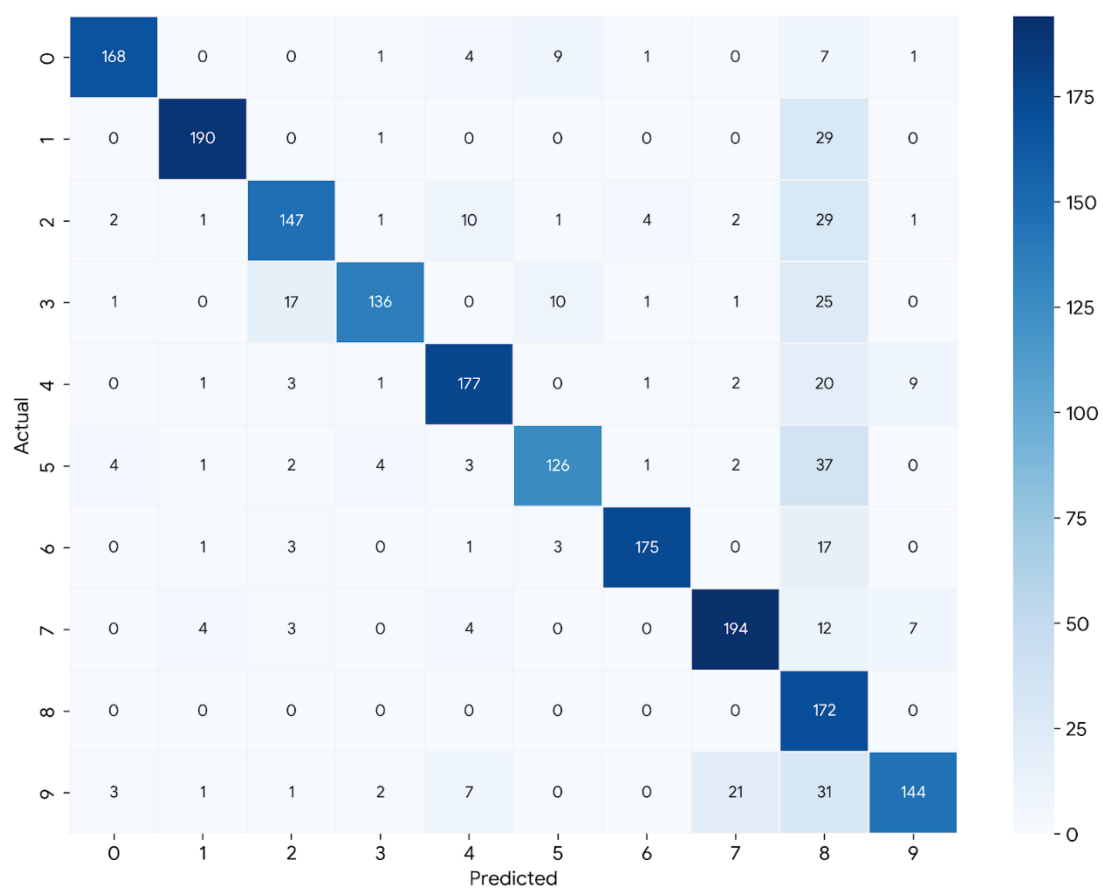
Real: 5, Pred: 1



learning_rate = 0.1

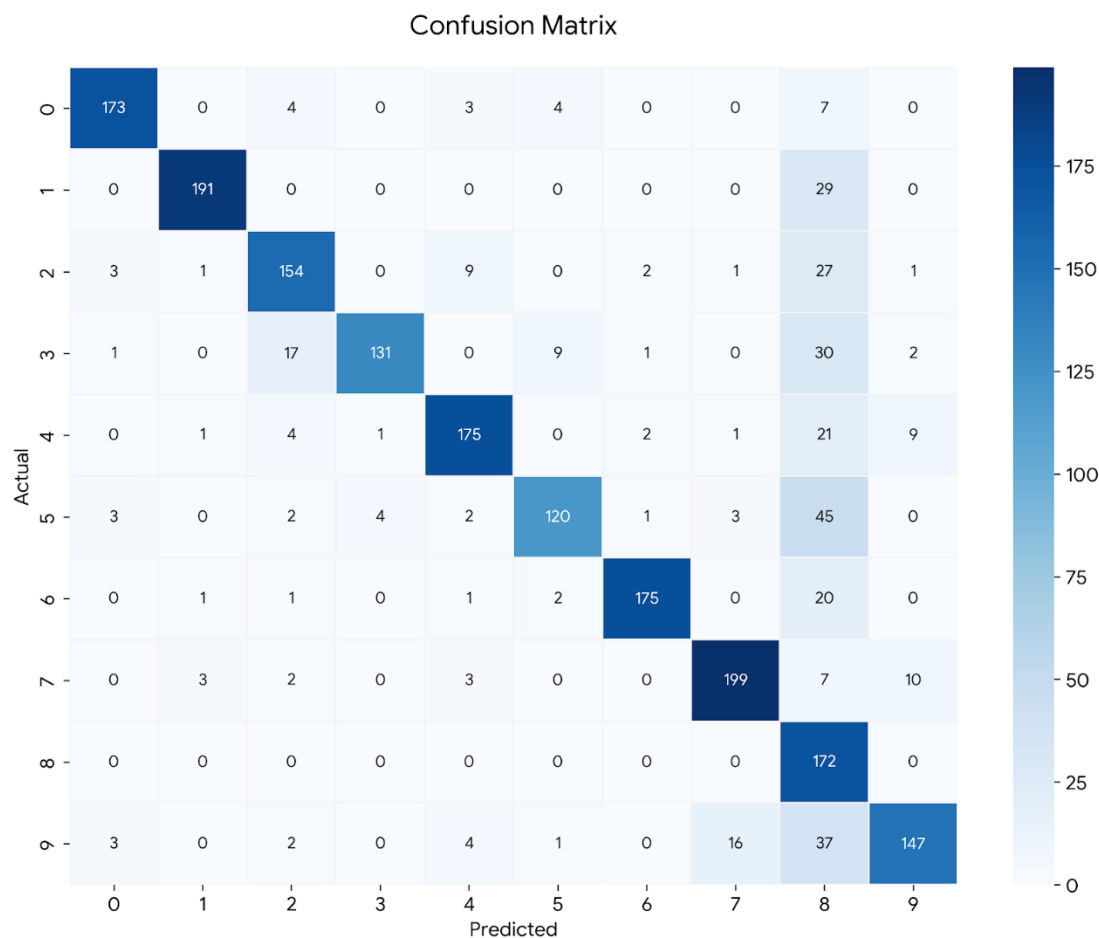
epochs = 10

Confusion Matrix



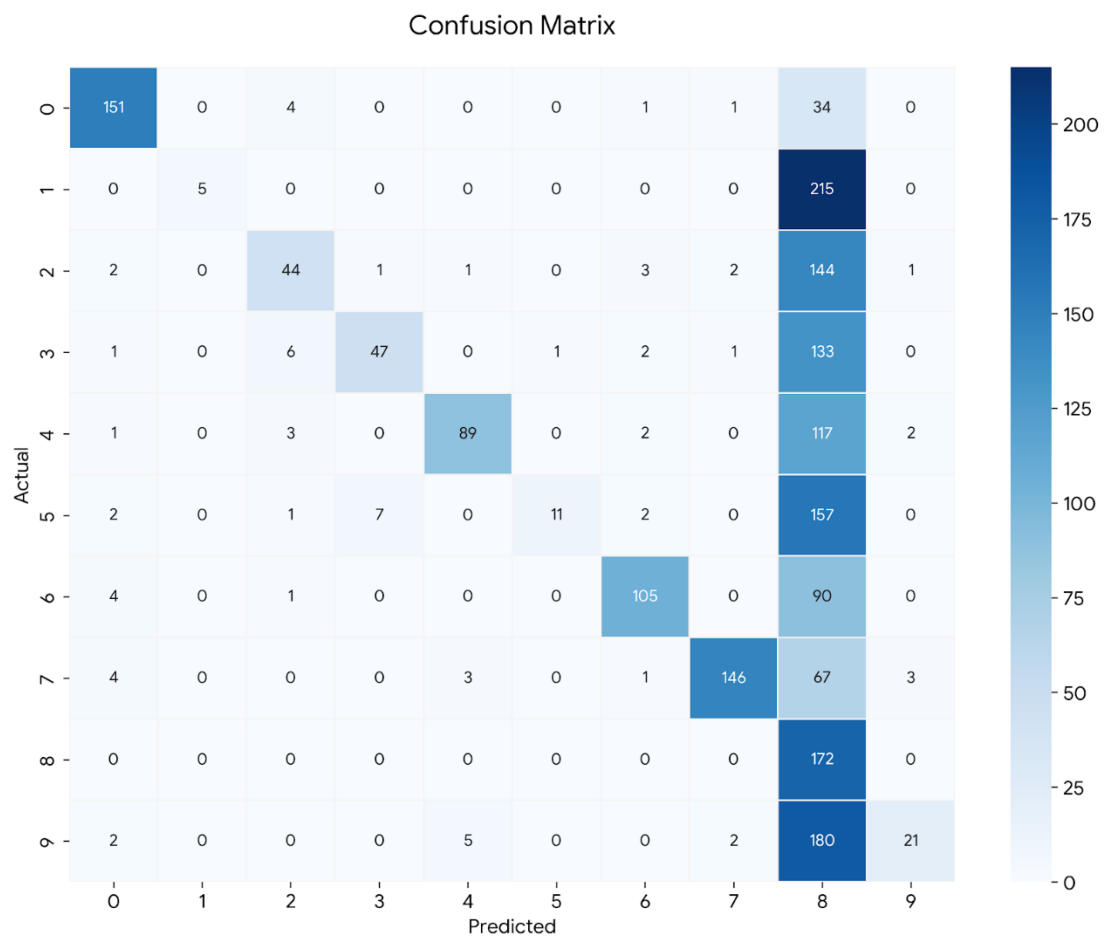
learning_rate = 0.001

epochs = 20



learning_rate = 0.0001

epochs = 30



Κρίνοντας από τα αποτελέσματα η πιο αποτελεσματική μέθοδος είναι η One vs One μιας και οι ταξινομητές παίρνουν πολύ μεγαλύτερη εξειδίκευση στην σύγκριση 2 μόνο αριθμών το οποίο έρχεται με το κόστος της ταχύτητας. Η προσθήκη της δυσανάλογης κατανομής δεδομένων μείωσε την ακρίβεια των αλγορίθμων κατά περίπου 200 επιτυχίες.